Stacklyst

Laboratório de Engenharia de Software



Processos de negócio, modelagem e funcionamento

Pedro Antonio Silvestre Tescaro
Guilherme Alves de Campos
Paulo Henrique Leal dos Santos

Seminário do projeto

Documento 1: visão do sistema

Problema atendido e proposta do Stacklyst

Problema

O estudo, a prática, as evidências de progresso e as oportunidades costumam ficar dispersos entre ferramentas.

Proposta

Reunir trilhas, lições e atividades de programação com retorno sobre as respostas e progresso confirmado.

Aplicação do aprendizado

Publicações, duelos, eventos e oportunidades conectam o estudo à comunidade e à prática profissional.

Públicos e necessidades

Documento 1: partes interessadas

Quem aprende

Organizar o estudo por linguagem, compreender os erros e acompanhar a evolução.

Comunidade e avaliadores

Compartilhar conhecimento e emitir pareceres com critérios e autorização.

Empresas e administração

Divulgar vagas, acompanhar candidaturas, manter usuários e tratar denúncias.

Escopo e critérios de sucesso

Documento 1: capacidades e fronteiras

Núcleo de aprendizagem

Consultar o percurso, realizar lições, resolver exercícios e acompanhar o desempenho.

Integridade do progresso

Distinguir prévia de submissão. Confirmar o resultado no servidor. Evitar duplicação de recompensas.

Autorizações e modelos

Verificar papel e vínculo por recurso. Usar os 26 casos da vista Caso de Uso Geral na especificação.

Ordem da apresentação

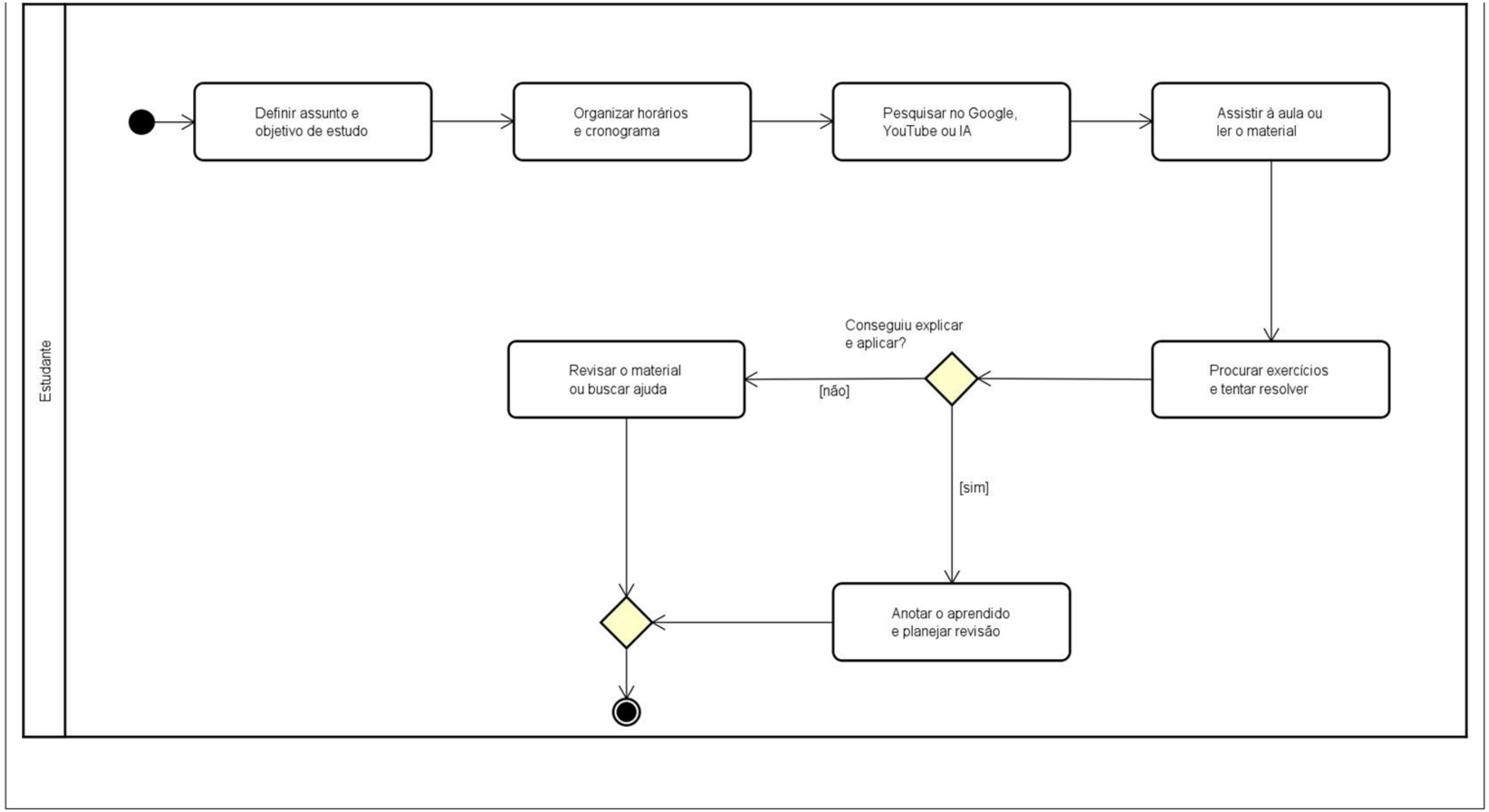
A continuidade entre os integrantes orienta a demonstração

Responsável	Funcionalidade e continuidade	Tempo
Grupo	Projeto, atividades do negócio e casos de uso gerais	9 min
Pedro	Lição e exercício, modelos gerais e progresso	13 min
Guilherme	Publicar a vaga que será usada na candidatura	12 min
Paulo	Candidatura, avanço da seleção e notificação	14 min
Grupo	Encerramento e perguntas	7 min

Planejamento de 55 minutos, com 5 minutos de margem até o limite de uma hora.

Estudar programação

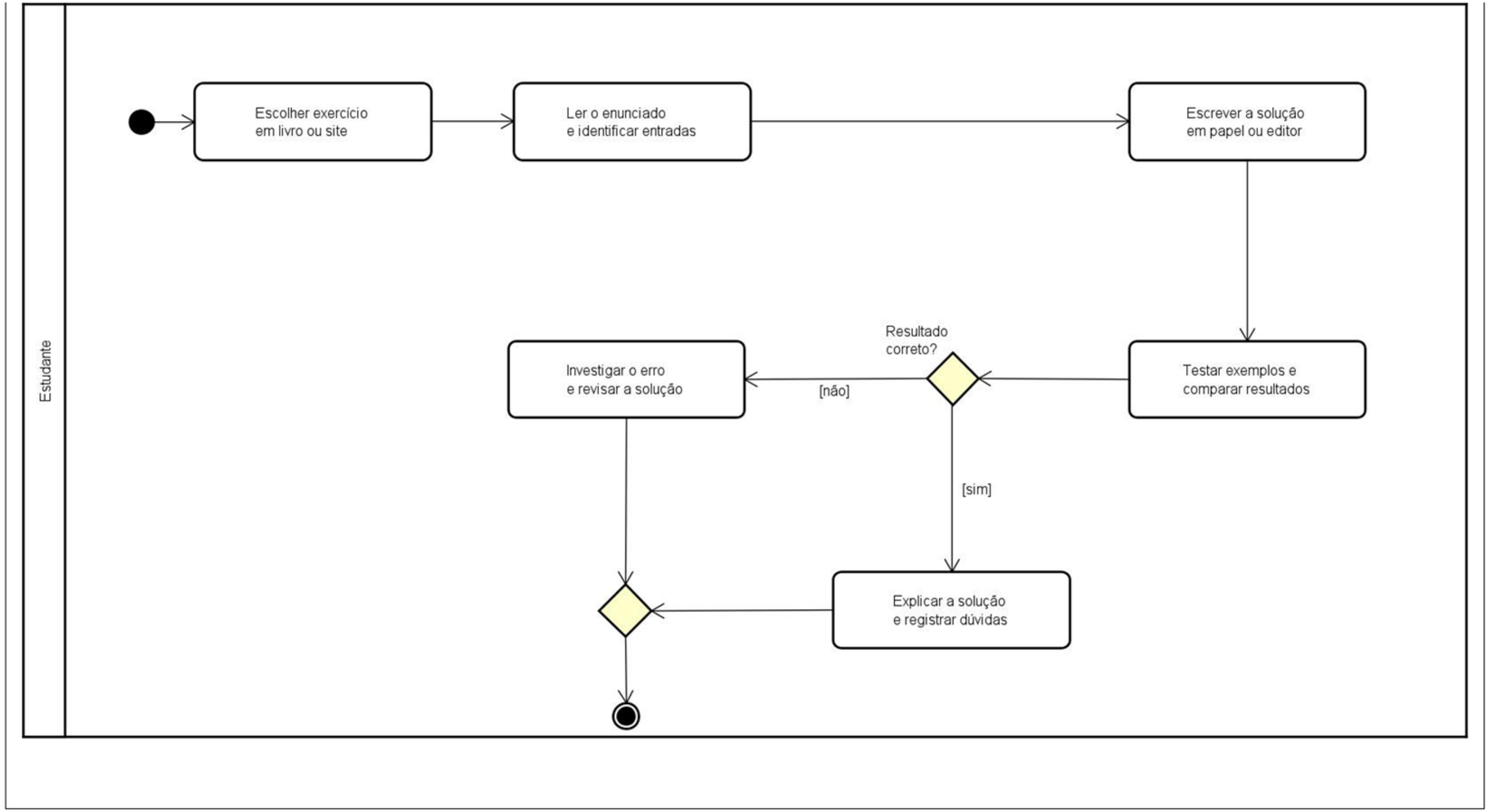
AN01 Atividades do negócio



Organizar o estudo e verificar se o conteúdo foi compreendido e aplicado.

Praticar exercícios de programação

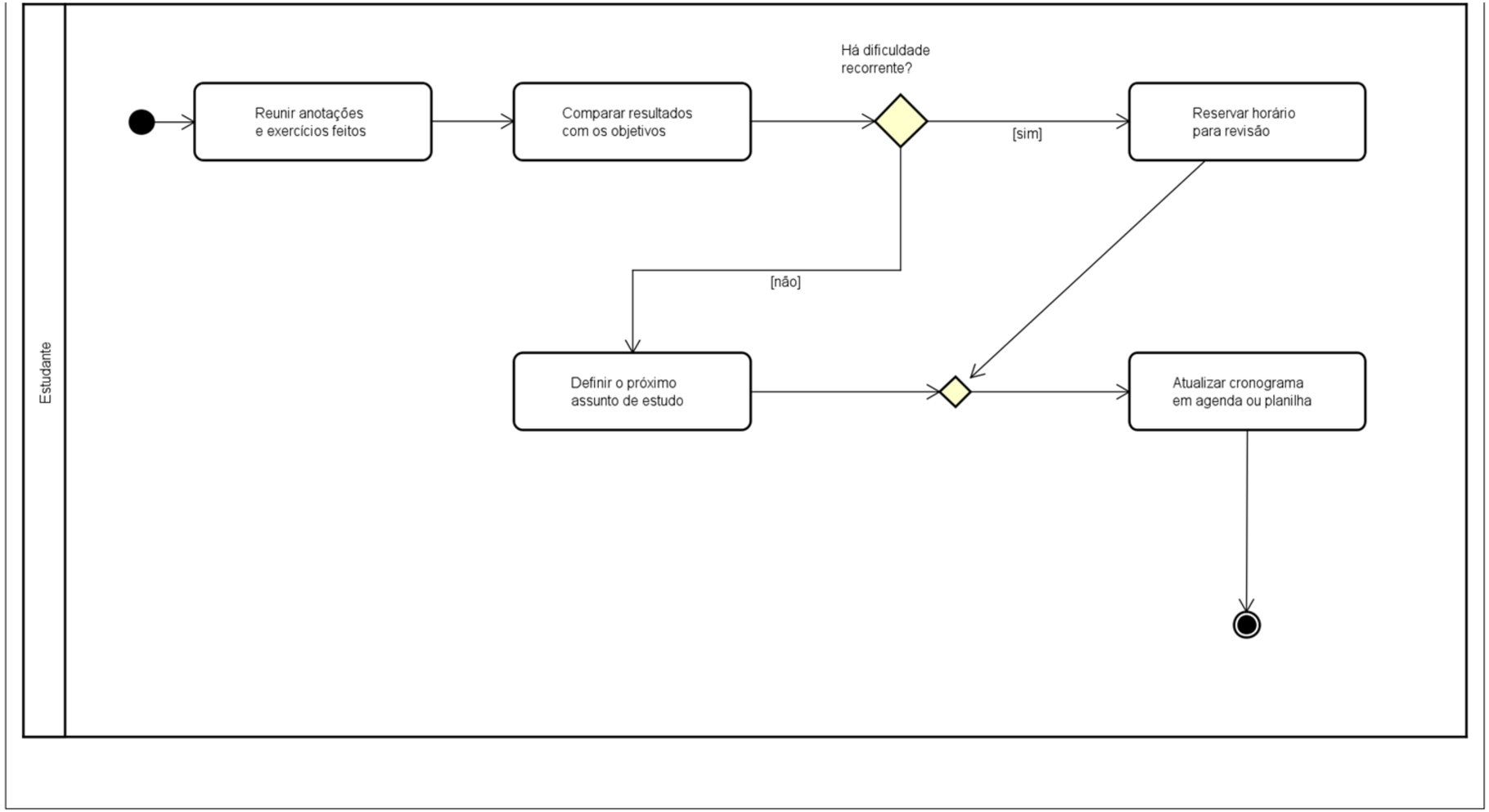
AN02 Atividades do negócio



Resolver um problema, testar exemplos e compreender os erros encontrados.

Acompanhar a própria aprendizagem

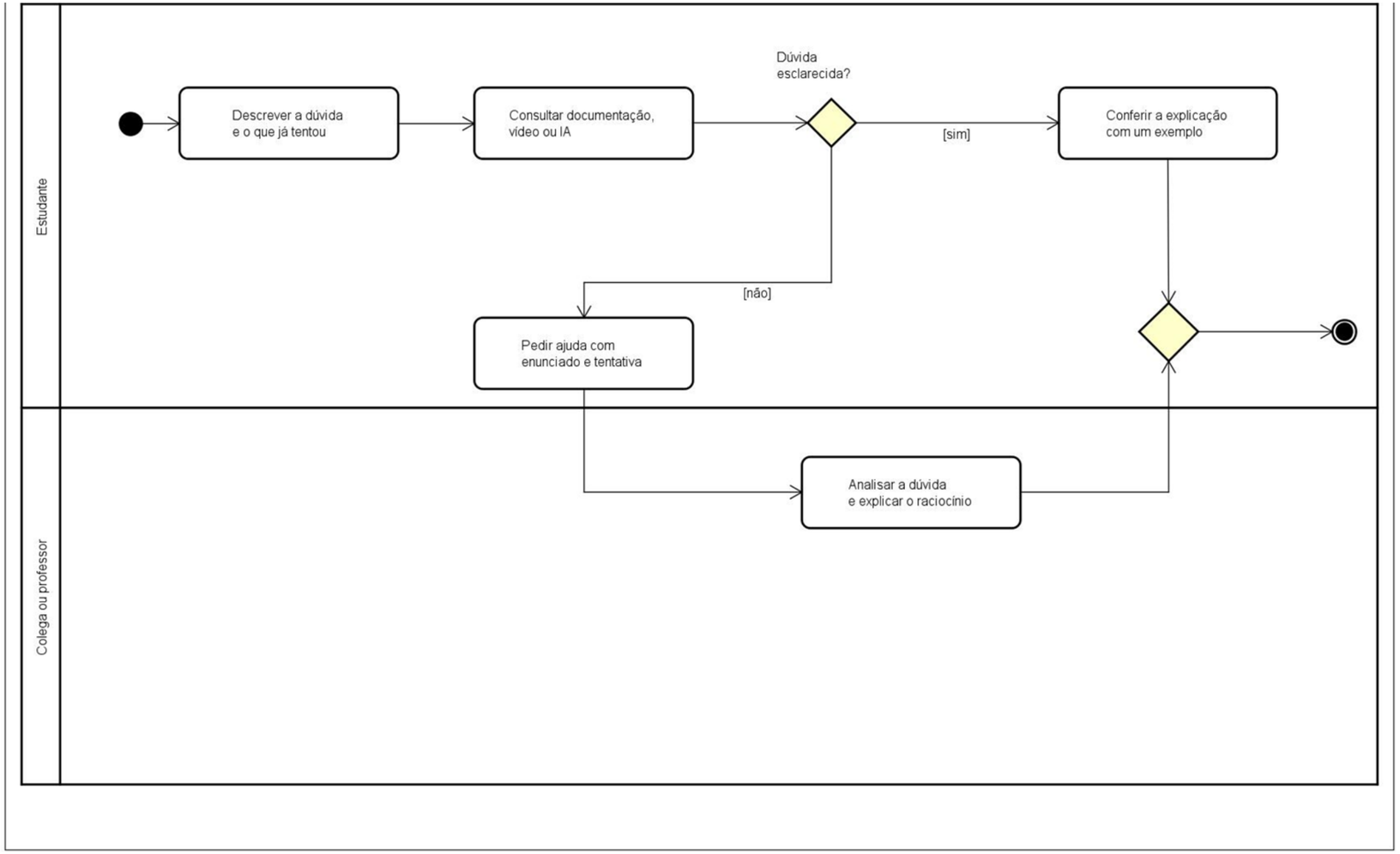
AN03 Atividades do negócio



Comparar os resultados aos objetivos e ajustar o planejamento de estudo.

Buscar ajuda para uma dúvida

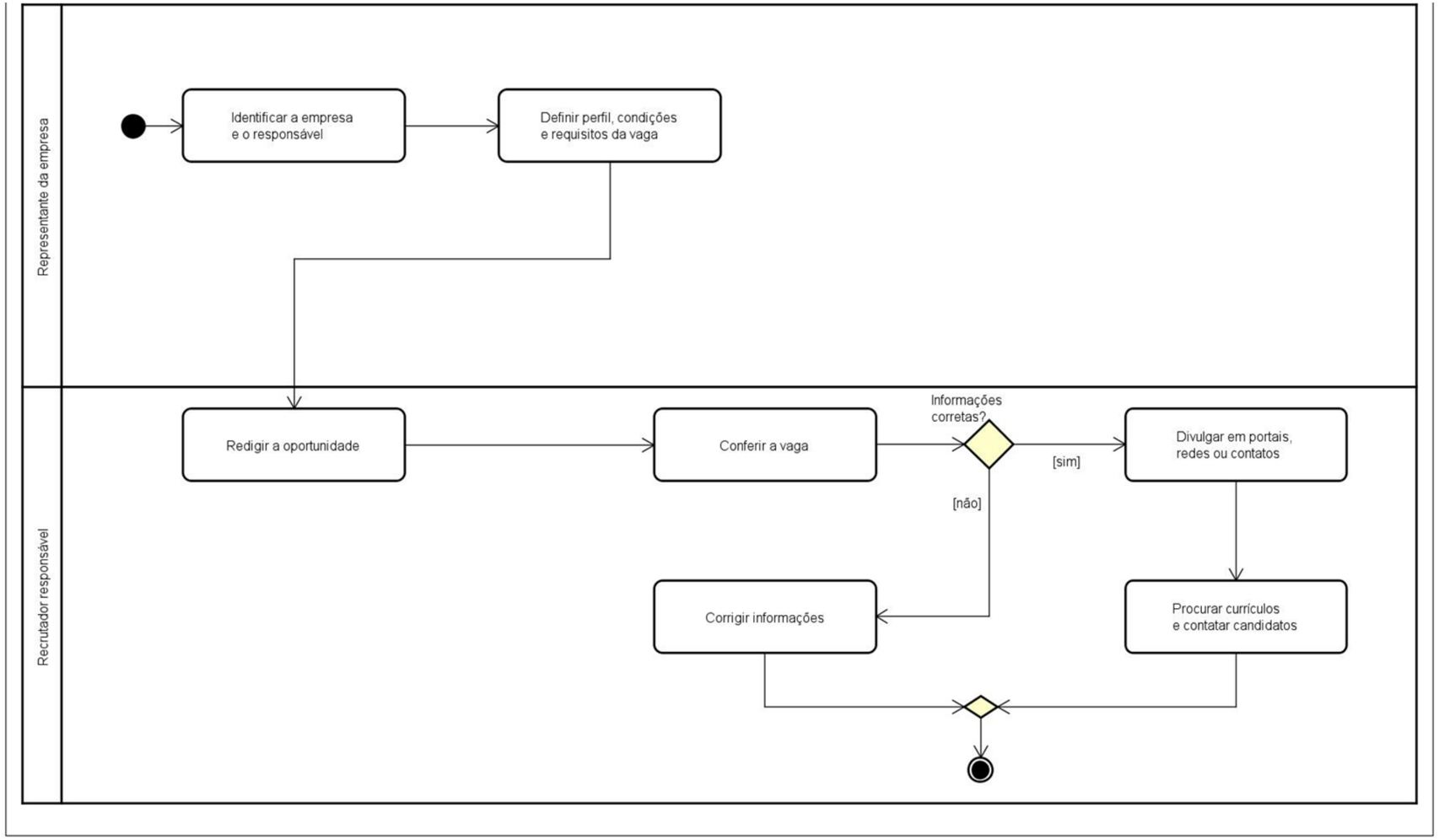
AN04 Atividades do negócio



Esclarecer uma dúvida e conferir a explicação em um exemplo.

Divulgar oportunidade de trabalho

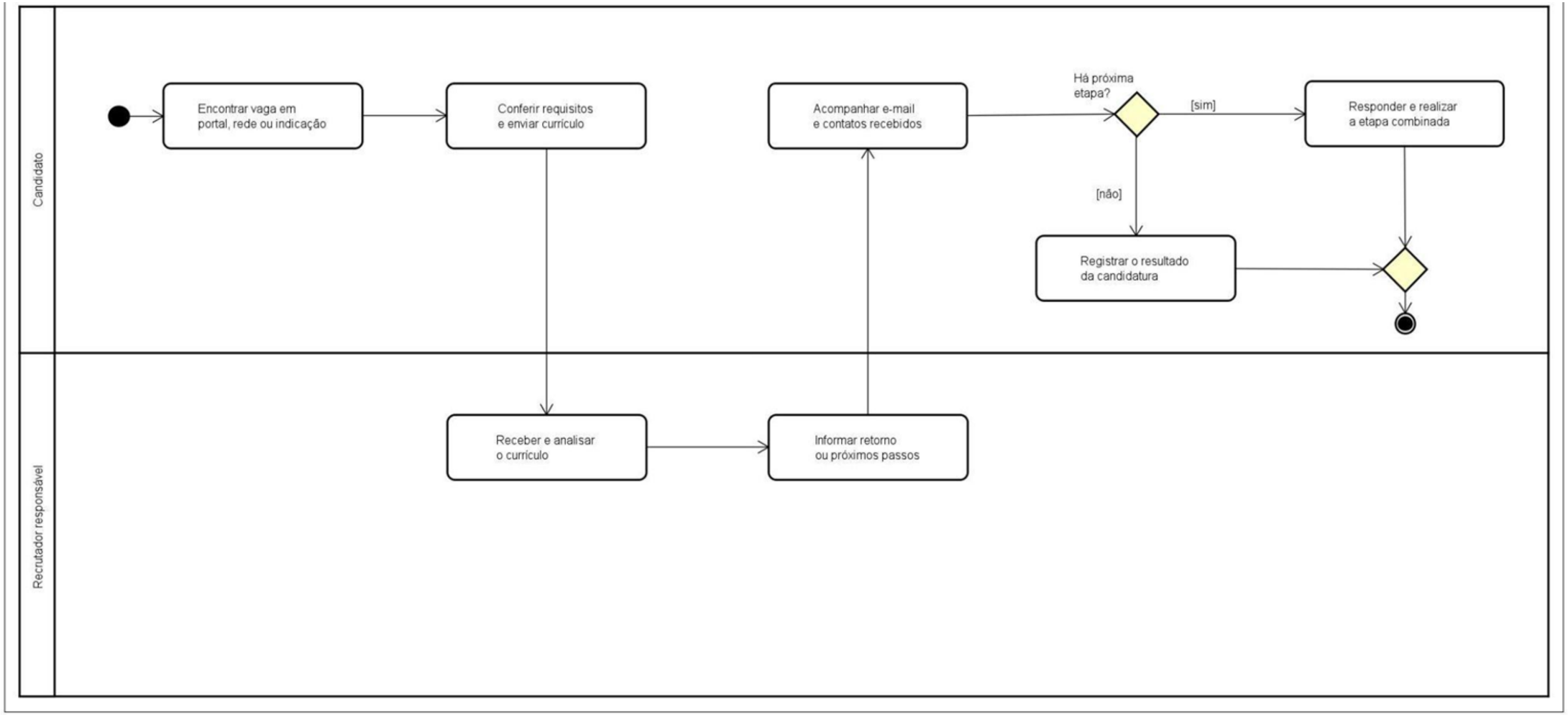
AN05A Atividades do negócio



Definir e divulgar uma oportunidade profissional com informações conferidas.

Candidatar-se e acompanhar a seleção

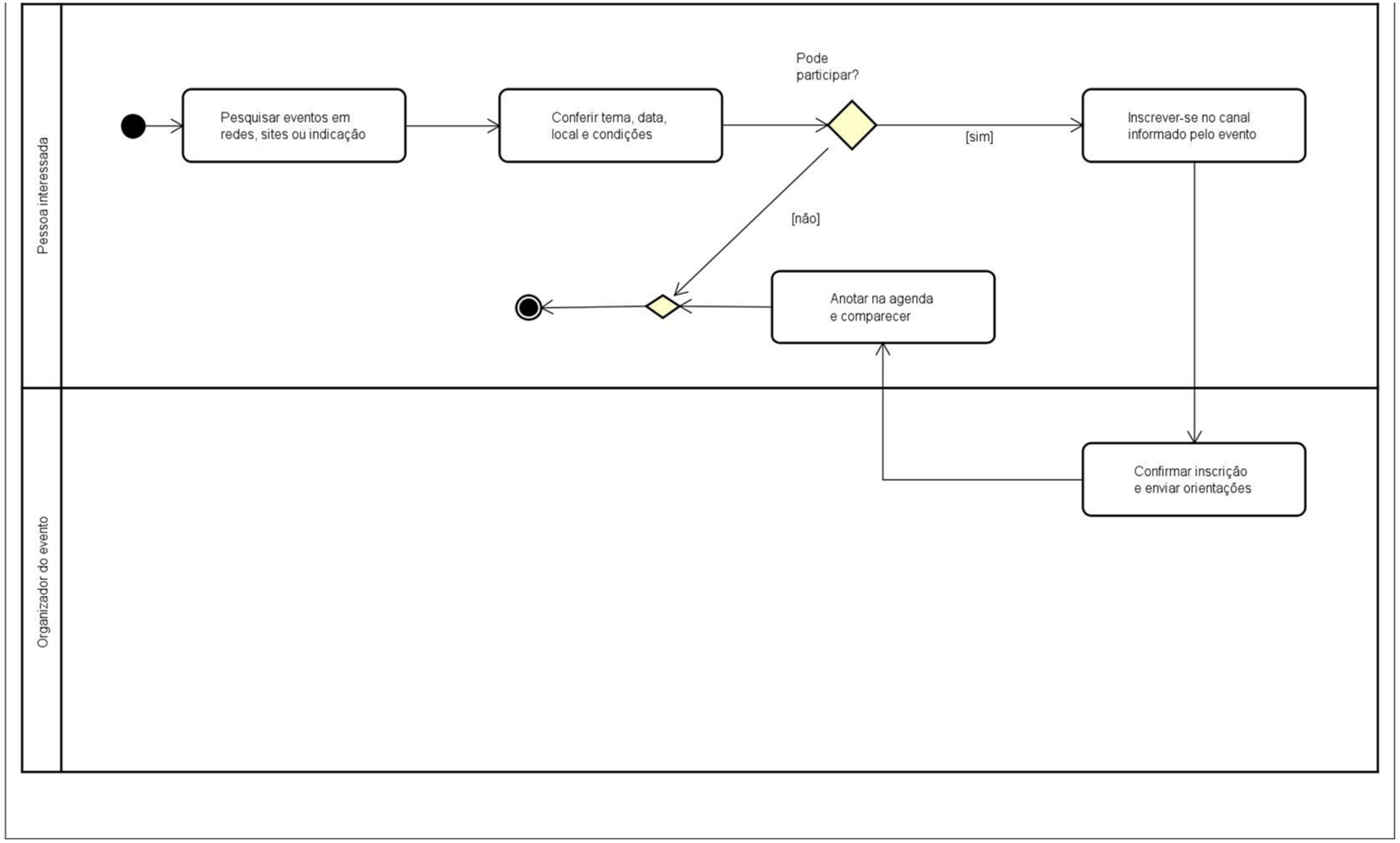
AN05B Atividades do negócio



Enviar o currículo e acompanhar as etapas e o resultado da seleção.

Descobrir e participar de um evento

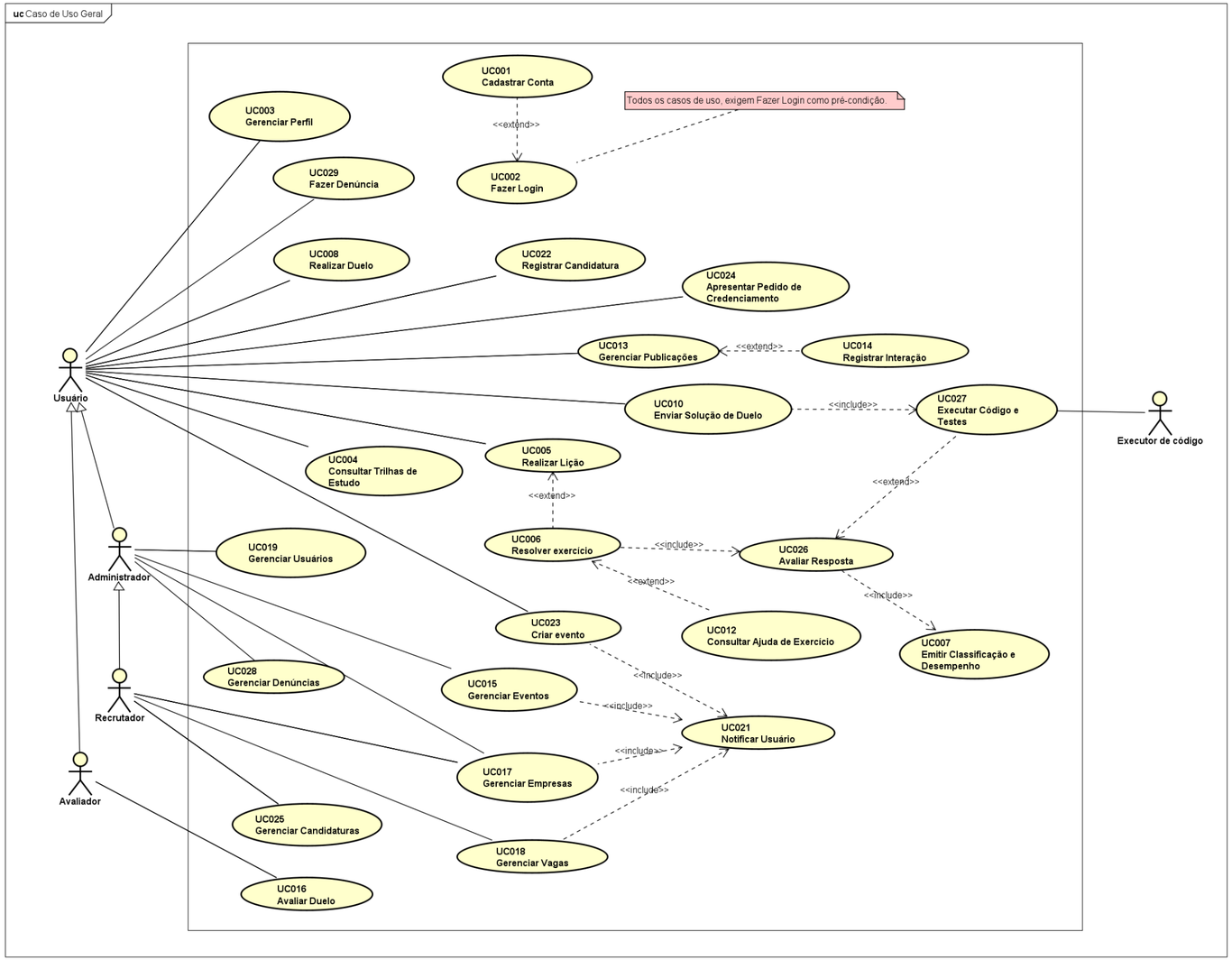
AN06 Atividades do negócio



Encontrar um evento e organizar a inscrição e a participação.

Caso de Uso Geral

Vista fornecida pelo grupo: nomes e IDs da especificação



Aprendizagem: fluxos da especificação

Fluxo principal e alternativas da mesma vista geral

UC005 Realizar Lição

1. O usuário abre uma lição disponível.
2. O sistema verifica a lição e o progresso correspondente.
3. A interface apresenta conceito, exemplo e orientação da etapa.
4. O usuário lê o conteúdo e avança às atividades de UC006.

Alternativa

A1 — Retomar uma lição: o sistema usa o progresso persistido e permite continuar ou revisar.

UC006 Resolver exercício

1. O usuário abre a atividade e consulta enunciado, opções ou editor.
2. O usuário informa uma resposta compatível com o tipo da atividade.
3. O servidor valida o contexto e solicita UC026 para avaliar a resposta.
4. A interface apresenta acerto, erro ou orientação aplicável.

Alternativa

A1 — Executar uma prévia: o código é testado, mas a prévia não conclui a atividade nem concede XP.

Recrutamento: fluxos da especificação

Fluxo principal e alternativas da mesma vista geral

UC022 Registrar Candidatura

1. O usuário consulta uma vaga e solicita a candidatura.
2. O servidor verifica a sessão, a existência da vaga e sua situação aberta.
3. O sistema verifica se já existe candidatura do mesmo usuário à vaga.
4. O sistema cria a candidatura com sua situação inicial e os vínculos corretos.

Alternativa

A1 — Candidatura já existente: o sistema informa a inscrição anterior e não cria outra.

UC025 Gerenciar Candidaturas

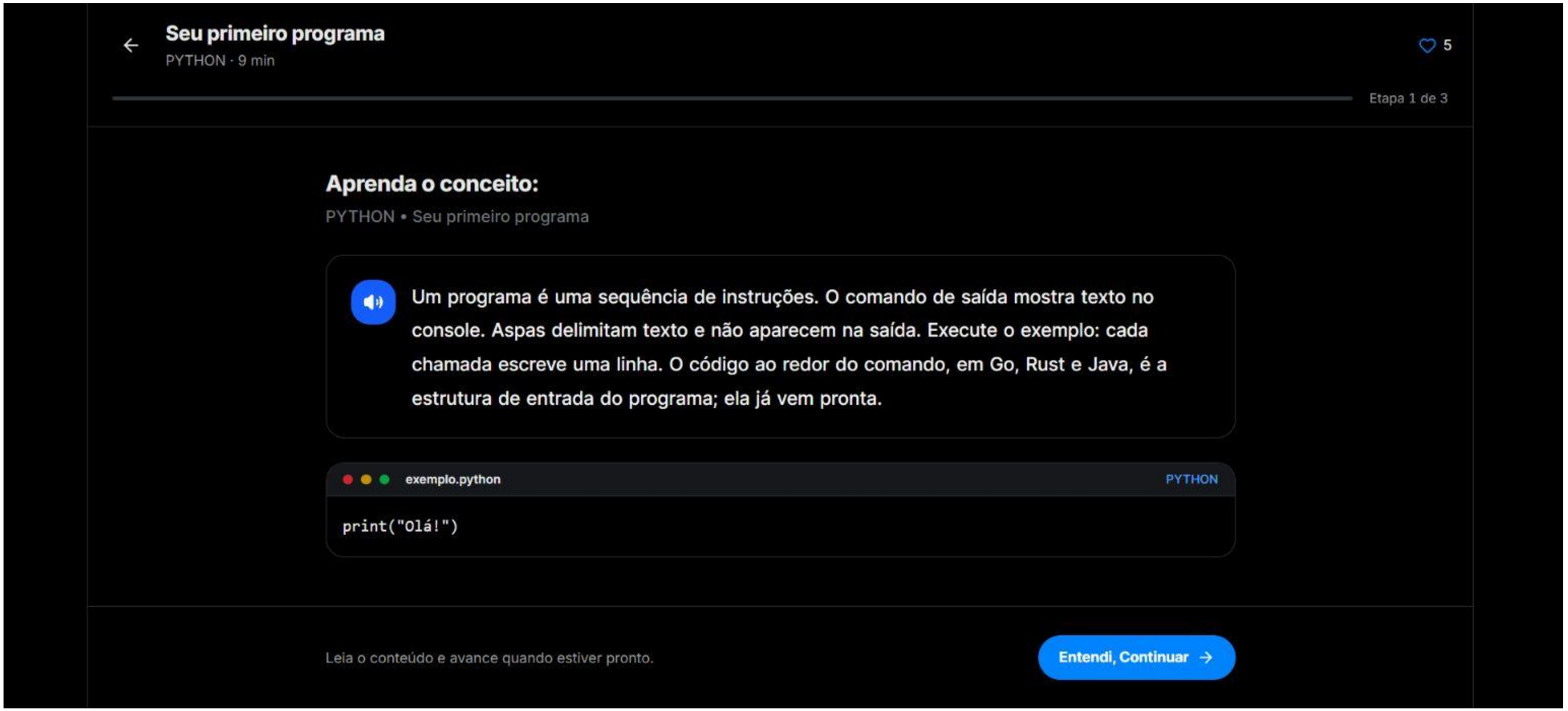
1. O gestor abre o painel de candidaturas de uma vaga autorizada.
2. O sistema valida o vínculo com a empresa e apresenta os candidatos.
3. O gestor seleciona uma candidatura e informa etapa, situação ou feedback.
4. O servidor verifica a compatibilidade entre candidatura, vaga e etapa.

Alternativa

A1 — Nova etapa, situação ou feedback: o sistema valida os vínculos, grava a mudança e registra a comunicação elegível.

Realizar Lição

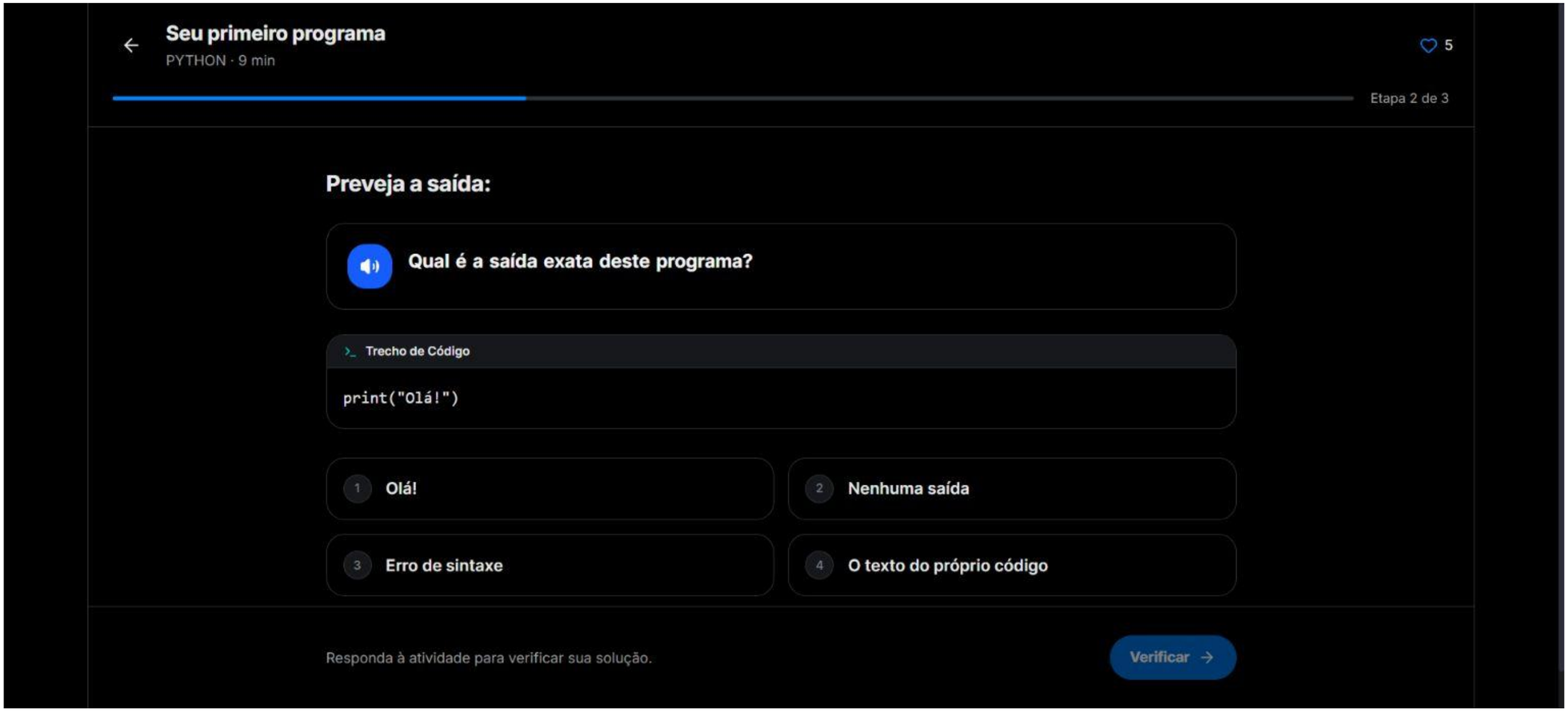
UC005: conteúdo e exemplo na interface atual



Interface atual da plataforma, capturada em 03/11/2026

Resolver exercício

UC006: enunciado e atividade na interface atual



Interface atual da plataforma, capturada em 03/11/2026

Requisitos e regras na interface

Aprendizagem

RF F06–F10

Abrir lição, responder, testar e submeter exercícios.

RNF Q01–Q05

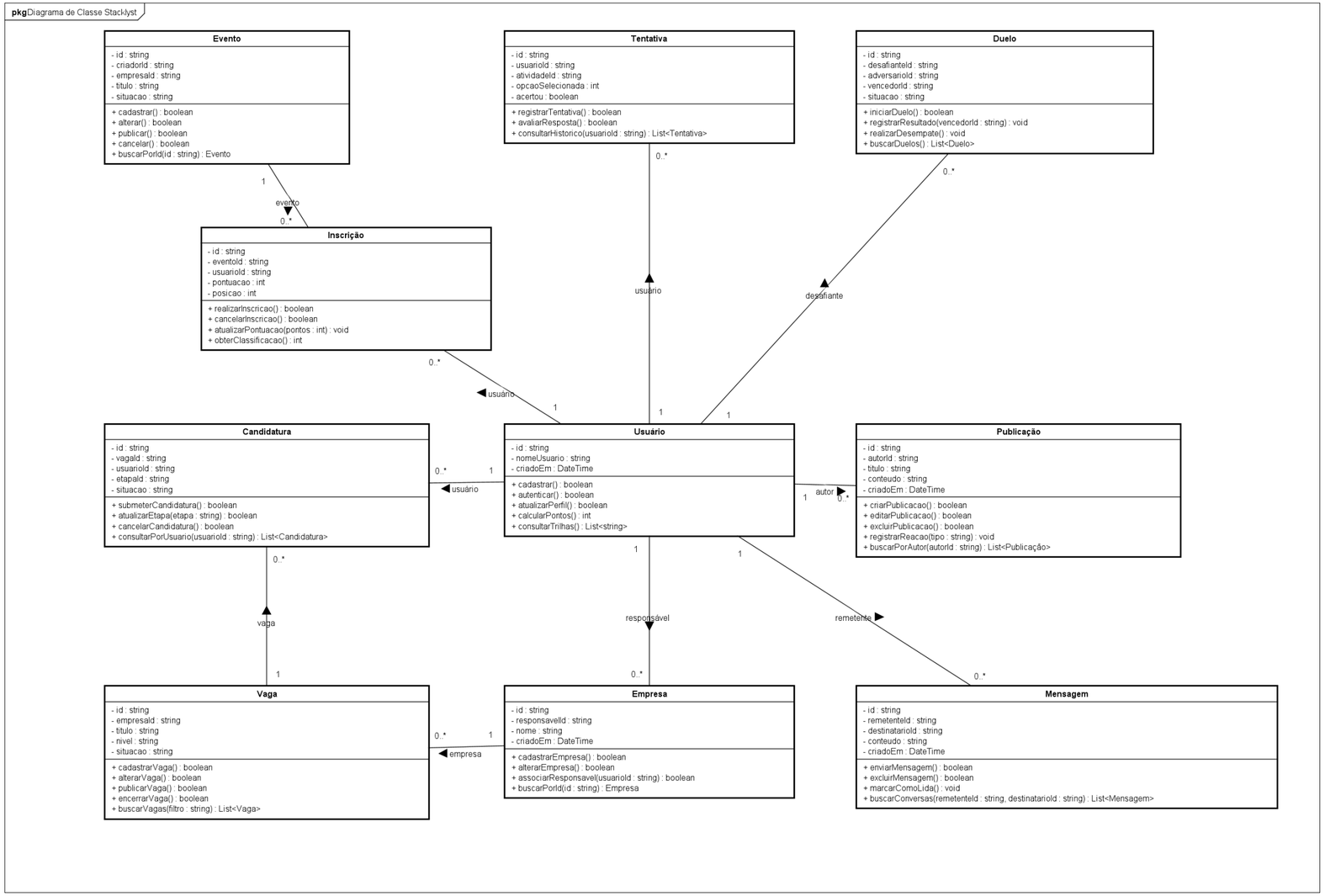
Autorizar acesso, preservar integridade e tratar falha do executor.

RN R12–R16

Avaliar no servidor. Prévia não concede XP. Repetição não duplica o crédito.

Classes principais do Stacklyst

Dez classes com atributos, métodos, parâmetros e retornos



Classes e implementação da aprendizagem

Responsabilidades e dados do percurso

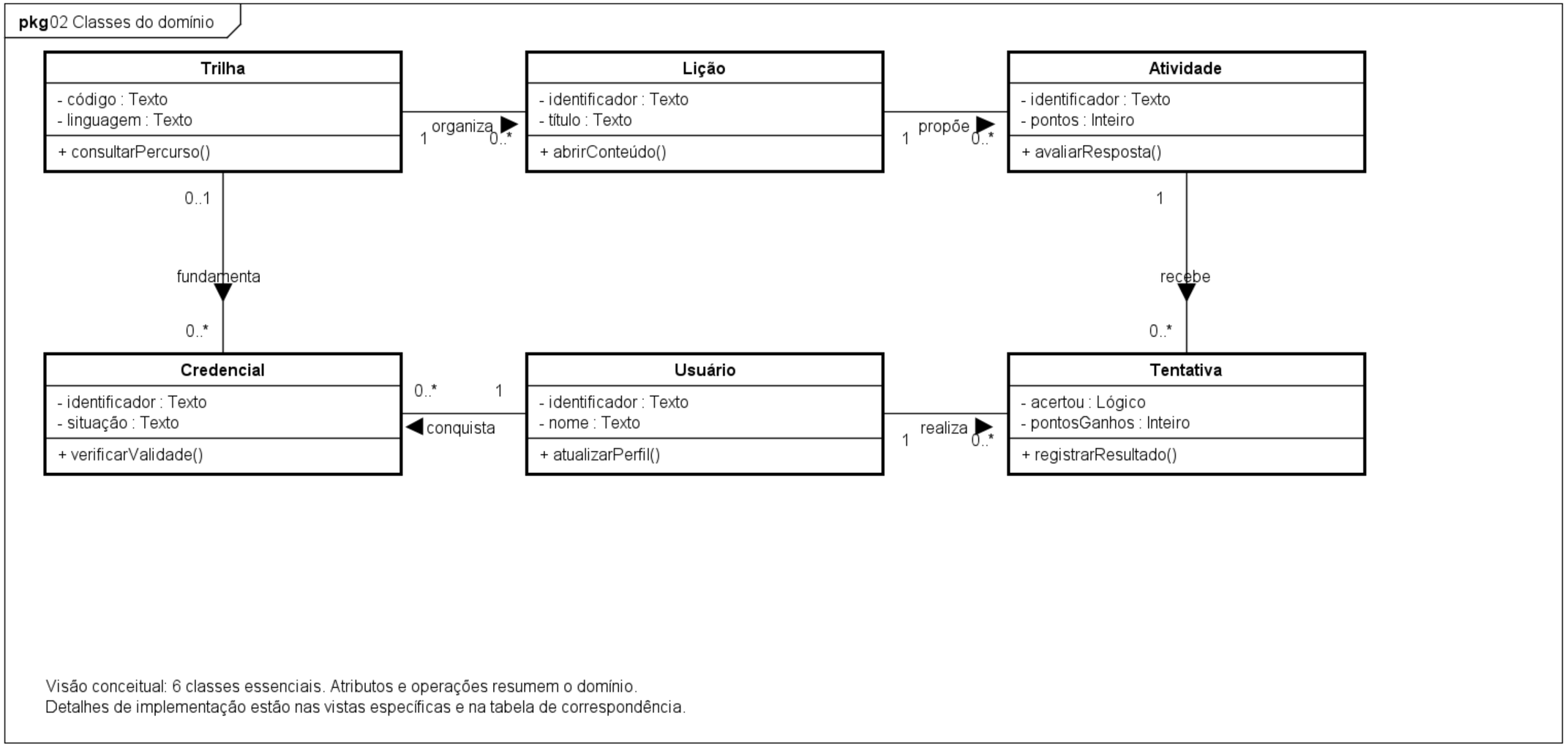
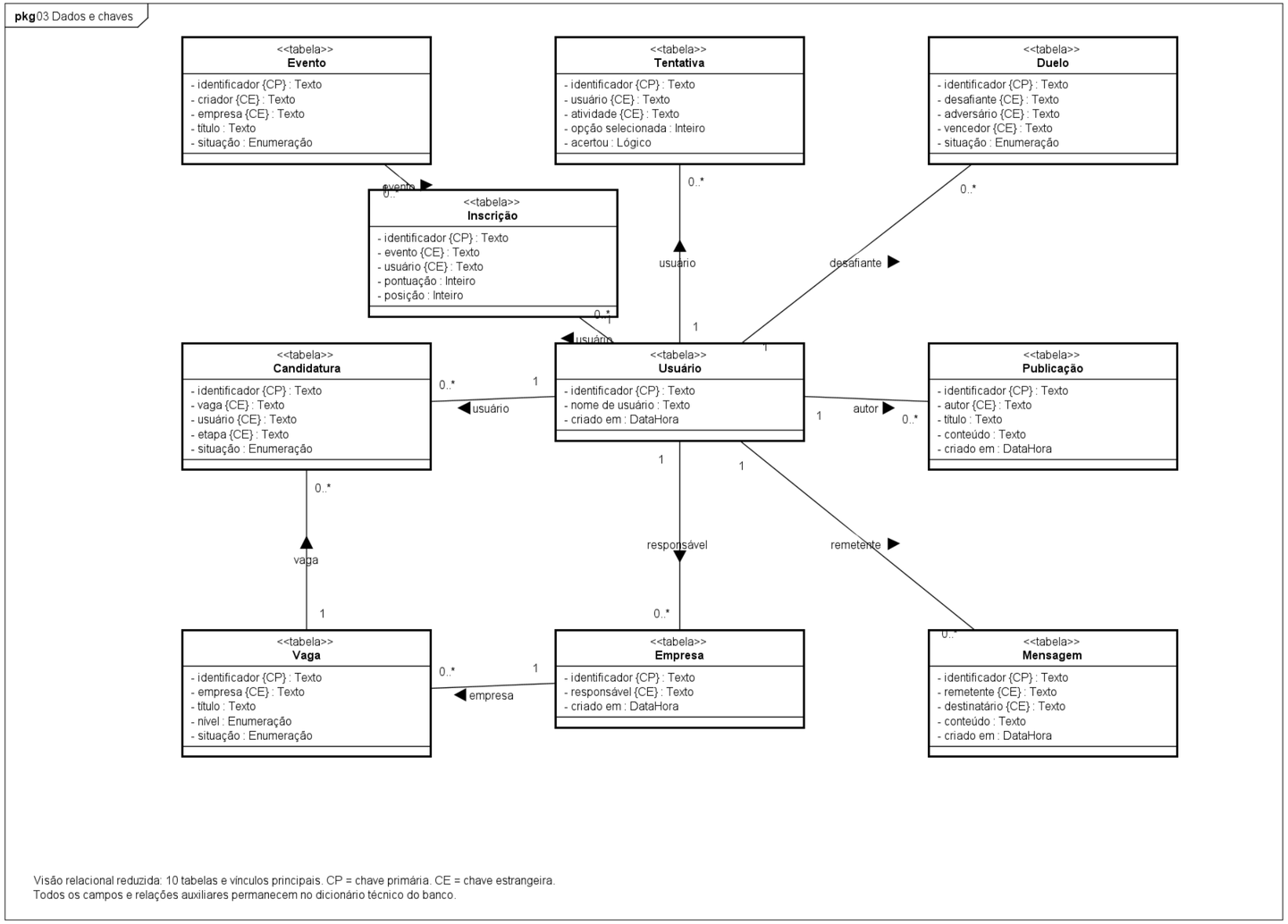


Diagrama geral do banco de dados

CP, CE e vínculos principais em notação UML



Visão relacional reduzida: 10 tabelas e vínculos principais. CP = chave primária. CE = chave estrangeira.
Todos os campos e relações auxiliares permanecem no dicionário técnico do banco.

Avaliar Resposta e Executar Código e Testes

Fluxo principal e alternativas da mesma vista geral

UC026 Avaliar Resposta

1. O sistema recebe a resposta e identifica a atividade e seu tipo.
2. O avaliador do servidor valida formato e contexto.
3. O sistema compara a resposta aos critérios ou aciona UC027 quando houver execução de código.
4. O sistema produz um resultado de acerto, falha ou indisponibilidade.

Alternativa

A1 — Atividade de código: a avaliação usa a execução e os testes de UC027 quando necessários.

UC027 Executar Código e Testes

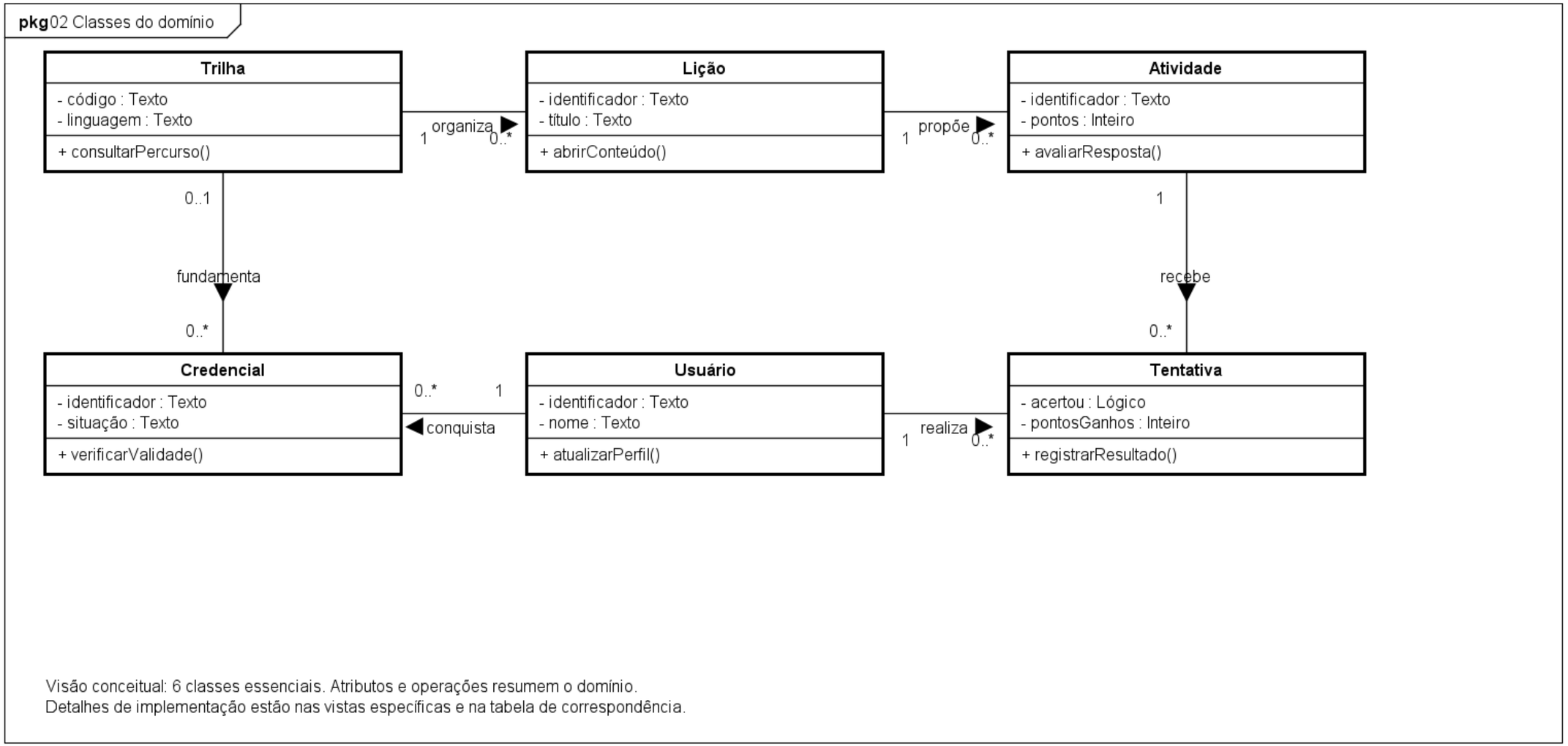
1. O sistema prepara a requisição autorizada de código, linguagem e testes.
2. O executor recebe a requisição e aplica os limites de execução.
3. O executor executa o código e verifica os casos de teste.
4. O executor devolve saída, erros, duração e resultado técnico.

Alternativa

A1 — Erro de sintaxe ou execução: a resposta diferencia falha do código de falha de infraestrutura.

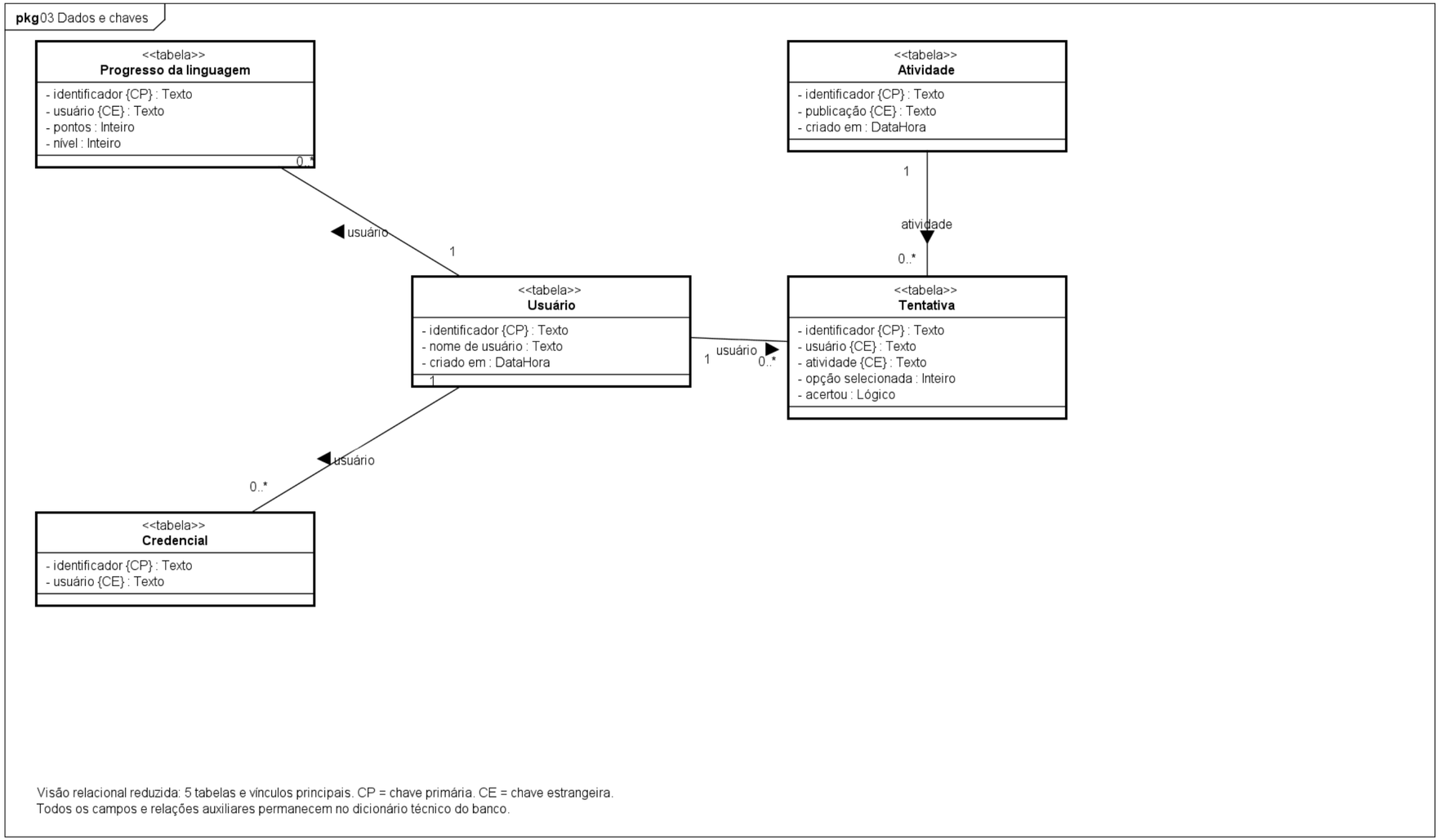
Classes da aprendizagem

Catálogo, progresso, tentativas e credenciais



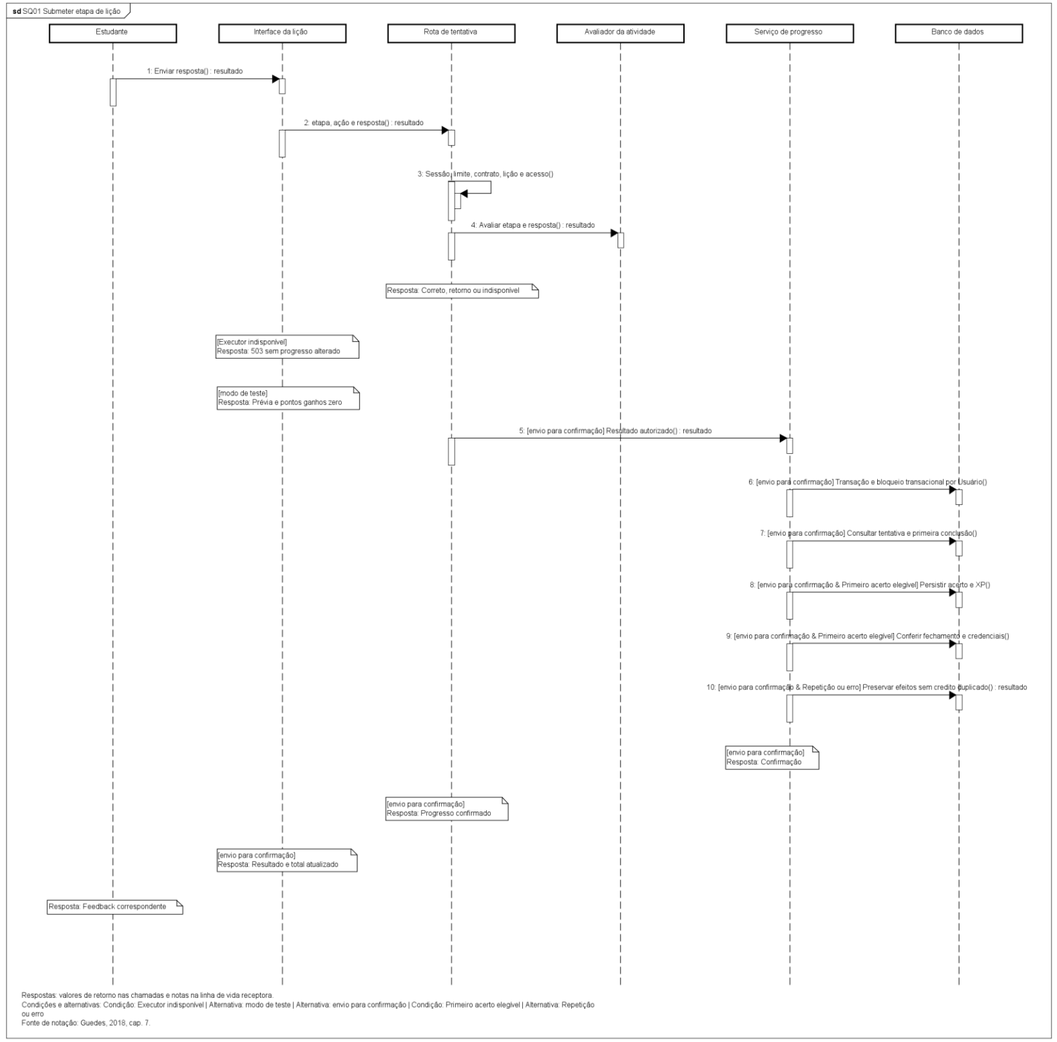
Banco de dados da aprendizagem

Chaves e registros persistidos



Sequência completa: resolver exercício

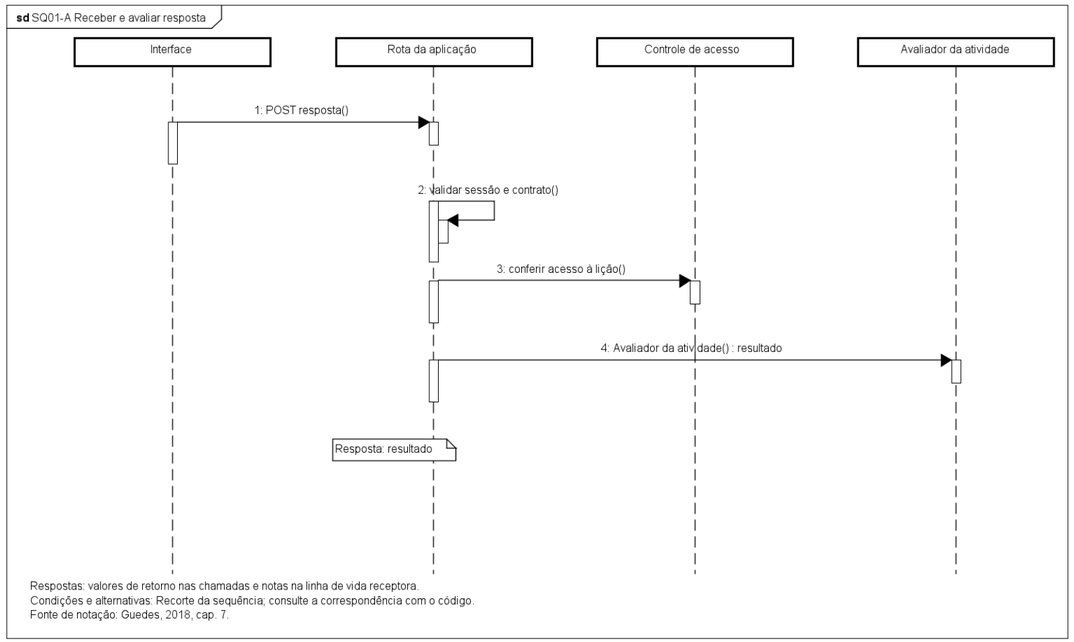
Envio, avaliação, prévia e confirmação



Receber e avaliar resposta

SQ01-A — sequência e código correspondente

Interação



Implementação

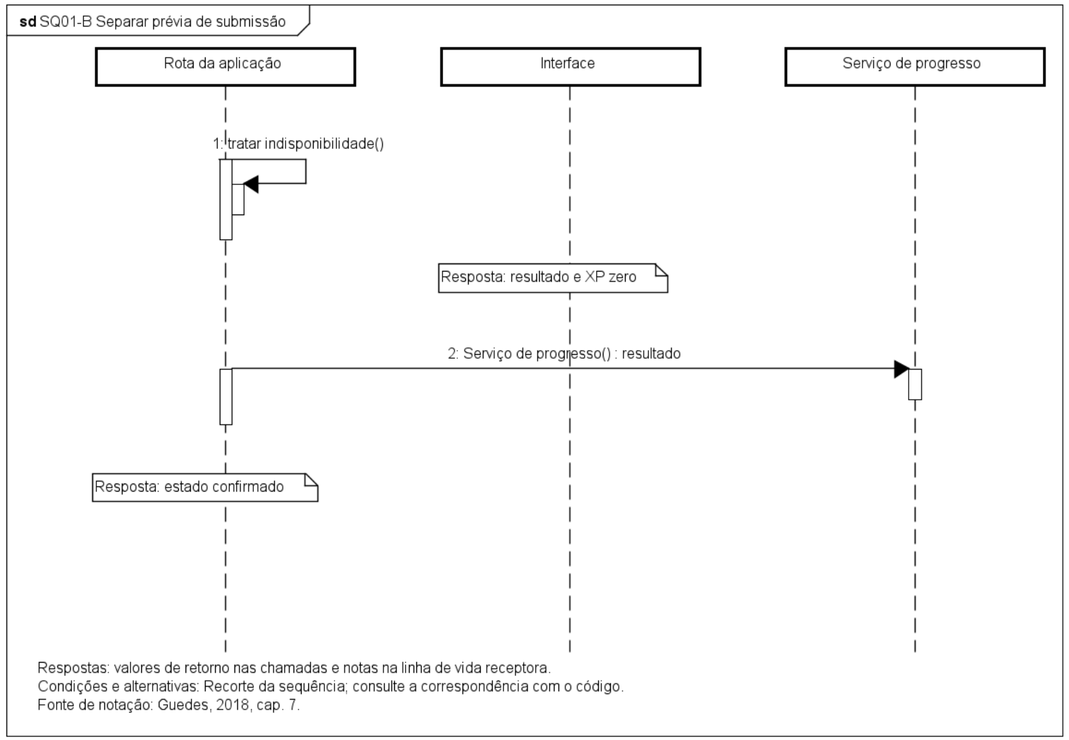
```
10  export const POST = apiHandler(async (request, { params, session: user }) => {
11    if (!user) throw new UnauthorizedError();
12    const { lessonId } = await params;
13    await rateLimit(`lesson:${user.id}`, {
14      limit: 30,
15      window: '1 m',
16      endpoint: '/api/lessons/attempt',
17    });
18    const parsed = lessonAnswerSchema.safeParse(await request.json().catch(() =>
null));
19    if (!parsed.success)
20      throw new ValidationError('INVALID_ANSWER', 'Confira os dados da resposta e
tente novamente.');
21    const answer = parsed.data;
22    const lesson = getLessonById(lessonId);
23    const step = lesson?.steps.find((s) => s.id === answer.stepId);
24    if (!lesson || !step) throw new AppError('LESSON_NOT_FOUND', 'Atividade não
encontrada.', 404);
25    await requireLessonAccess(user.id, lesson);
26    const outcome = await assessLessonStep(lesson, step, answer);
27    if ('unavailable' in outcome && outcome.unavailable)
28      throw new AppError(
29        'RUNNER_UNAVAILABLE',
30        'Serviço de execução indisponível. Seu progresso não foi alterado; tente
novamente.',
31        503
```

src/app/api/lessons/[lessonId]/attempt/route.ts Linhas 10–31

Separar prévia de submissão

SQ01-B — sequência e código correspondente

Interação



Implementação

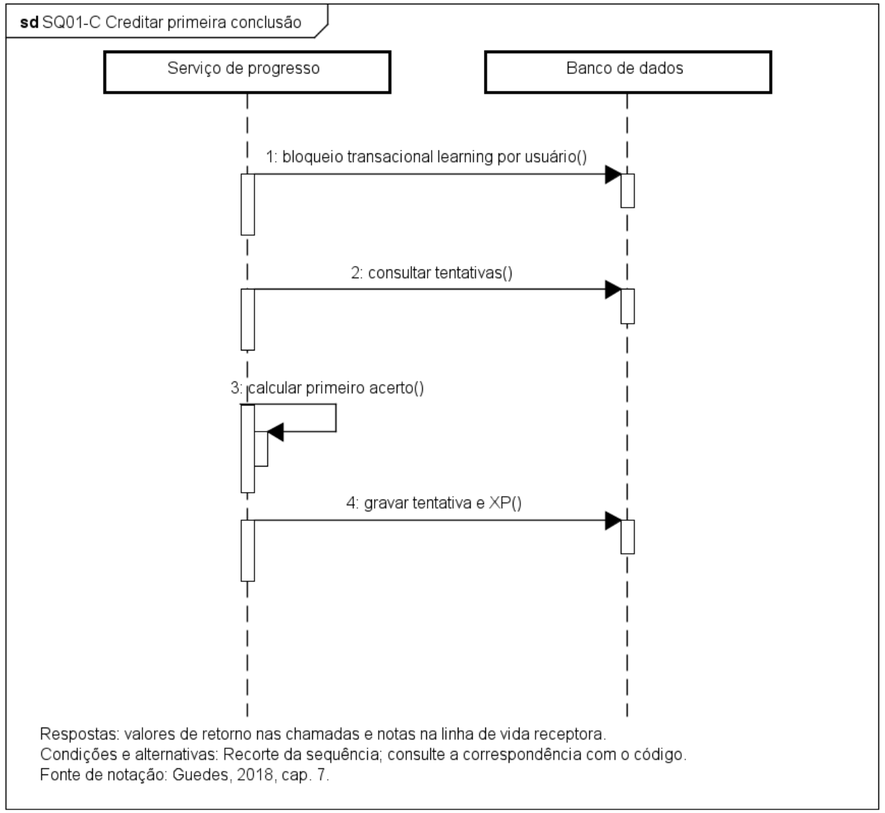
```
  33    if (answer.action === 'run') return NextResponse.json({ ...outcome, xpEarned: 0
});
  34    const progress = await saveLessonAssessment(
  35      user.id,
  36      lesson,
  37      step,
  38      outcome.isCorrect,
  39      answer.selectedOption ?? 0
  40    );
  41    if (outcome.isCorrect) {
  42      await syncProfileAchievementsSafely(user.id);
  43    }
  44    return NextResponse.json({ ...outcome, ...progress });
  45  });
```

src/app/api/lessons/[lessonId]/attempt/route.ts Linhas 33–45

Creditar primeira conclusão

SQ01-C — sequência e código correspondente

Interação



Implementação

```
62      const firstCompletion = isCorrect && !previous?.is_correct;
63      const xpEarned = firstCompletion ? Math.max(0, step.xp) : 0;
64      if (!previous)
65        await tx.quiz.upsert({
66          where: { id: step.id },
67          create: {
68            id: step.id,
69            question: step.question ?? step.instruction ?? step.title,
70            options: step.options ?? ['Avaliado no servidor'],
71            correct_index: step.correctOptionIndex ?? 0,
72            is_daily: false,
73          },
74          update: {},
75        });
76      if (!previous)
77        await tx.quizAttempt.create({
78          data: {
79            user_id: userId,
80            quiz_id: step.id,
81            selected_index: selectedIndex,
82            is_correct: isCorrect,
83            xp_earned: xpEarned,
```

src/lib/learning/lesson-progress.ts Linhas 62–83

Demonstração prática

Pedro Antonio Silvestre Tescaro Aprendizagem e conclusão de atividades da lição

Preparação

Usar conta de demonstração com progresso conhecido e anotar o XP inicial. Deixar aberta a lição de catálogo /lesson/js-l1, sem o parâmetro mode=workspace.

Execução

Mostrar o enunciado e a resposta preparada da atividade da lição. Acionar Testar e explicar que a resposta é avaliada no servidor sem conceder XP.

Resultado esperado

Ação run retorna xpEarned igual a zero. A conclusão correta inédita pode conceder o XP da etapa.

Conferência

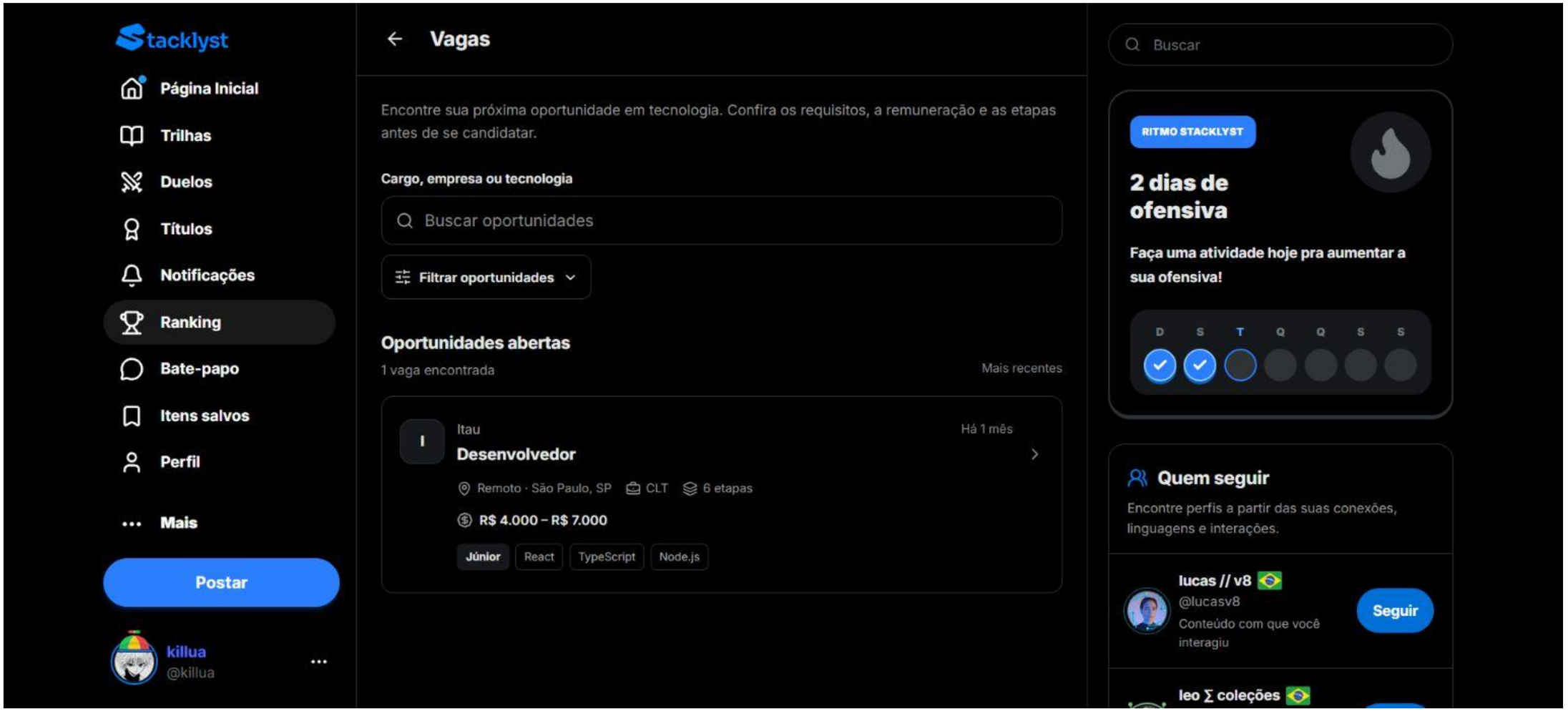
Acionar a verificação definitiva da solução e observar o resultado e o progresso. Comparar o XP ao valor anterior e repetir a verificação para mostrar que a mesma conclusão não concede XP novamente.

Continuidade

O perfil do candidato já existe. A próxima demonstração cria a oportunidade que será consumida no último bloco.

Guilherme: Gerenciar Vagas

UC018: consulta atual das oportunidades



Interface atual da plataforma, capturada em 03/11/2026

Requisitos e regras na interface

Empresa e vagas

RF F38–F39

Gerenciar empresa autorizada e publicar oportunidades.

RNF Q01, Q08, Q12

Conferir papel e vínculo. Informar confirmação ou erro.

RN R02, R04

Permissão é verificada por empresa e por recurso.

Gerenciar Empresas e Gerenciar Vagas

Fluxo principal e alternativas da mesma vista geral

UC017 Gerenciar Empresas

1. O gestor abre a área de empresas e informa ou seleciona a empresa.
2. O sistema confere seu papel e o vínculo de responsabilidade.
3. O gestor informa ou atualiza os dados cadastrais permitidos.
4. O sistema valida e grava o cadastro da empresa.

Alternativa

A1 — Atualizar empresa existente: o sistema confere o vínculo do gestor antes de alterar os dados.

UC018 Gerenciar Vagas

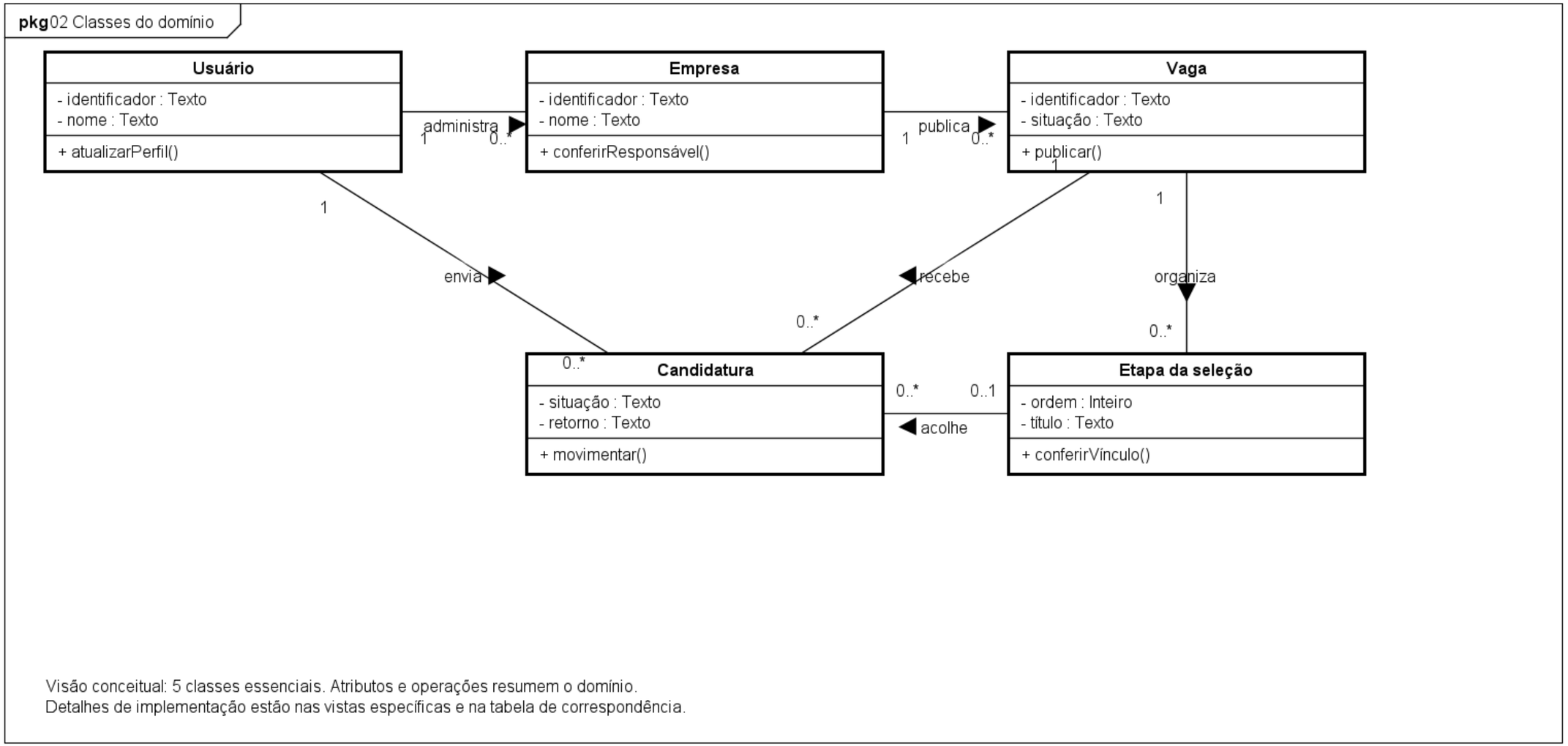
1. O gestor seleciona uma empresa autorizada e abre o cadastro ou a edição da vaga.
2. O gestor informa título, descrição, tecnologias, modalidade e etapas.
3. O servidor verifica o vínculo com a empresa e valida a oportunidade.
4. O sistema grava a vaga e suas etapas de recrutamento.

Alternativa

A1 — Alterar, encerrar ou filtrar uma vaga: o sistema aplica a autorização e a situação atual do registro.

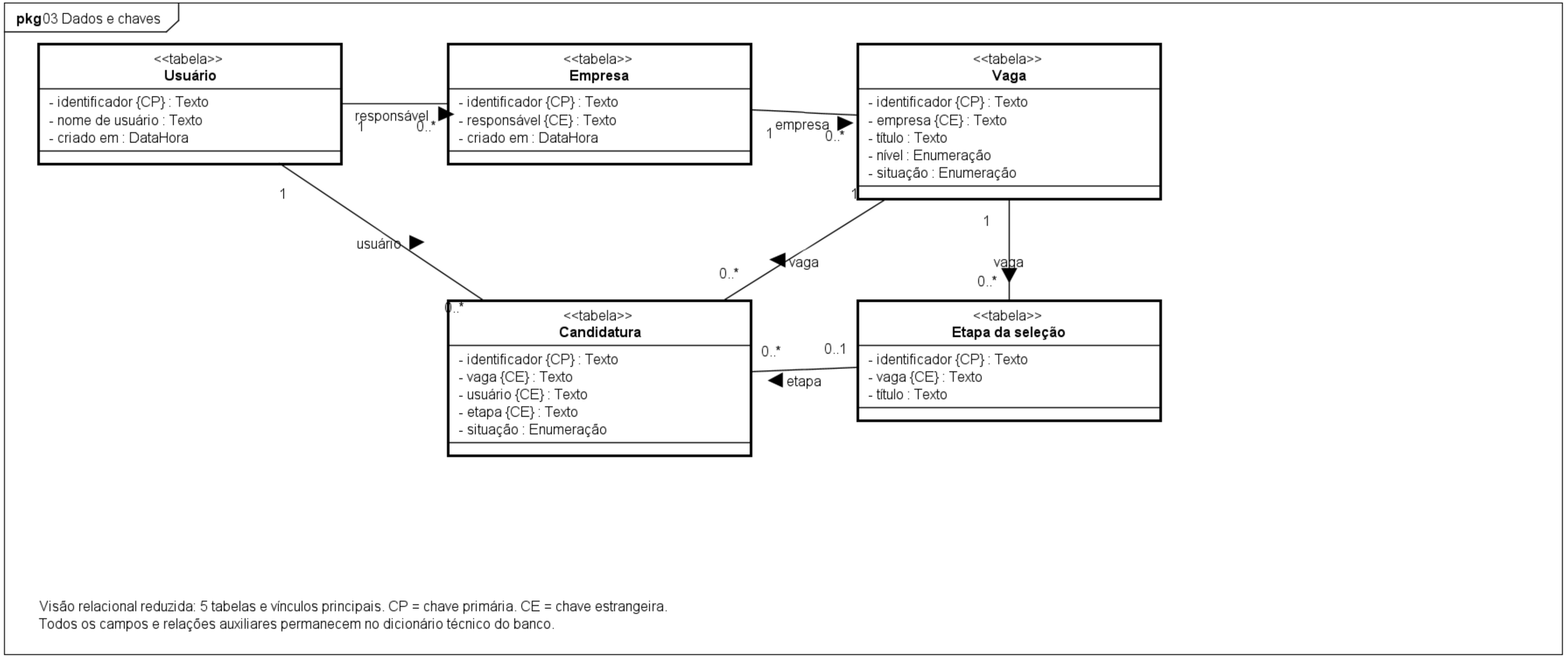
Classes da publicação de vaga

Empresa, oportunidade e etapas



Banco de dados da publicação

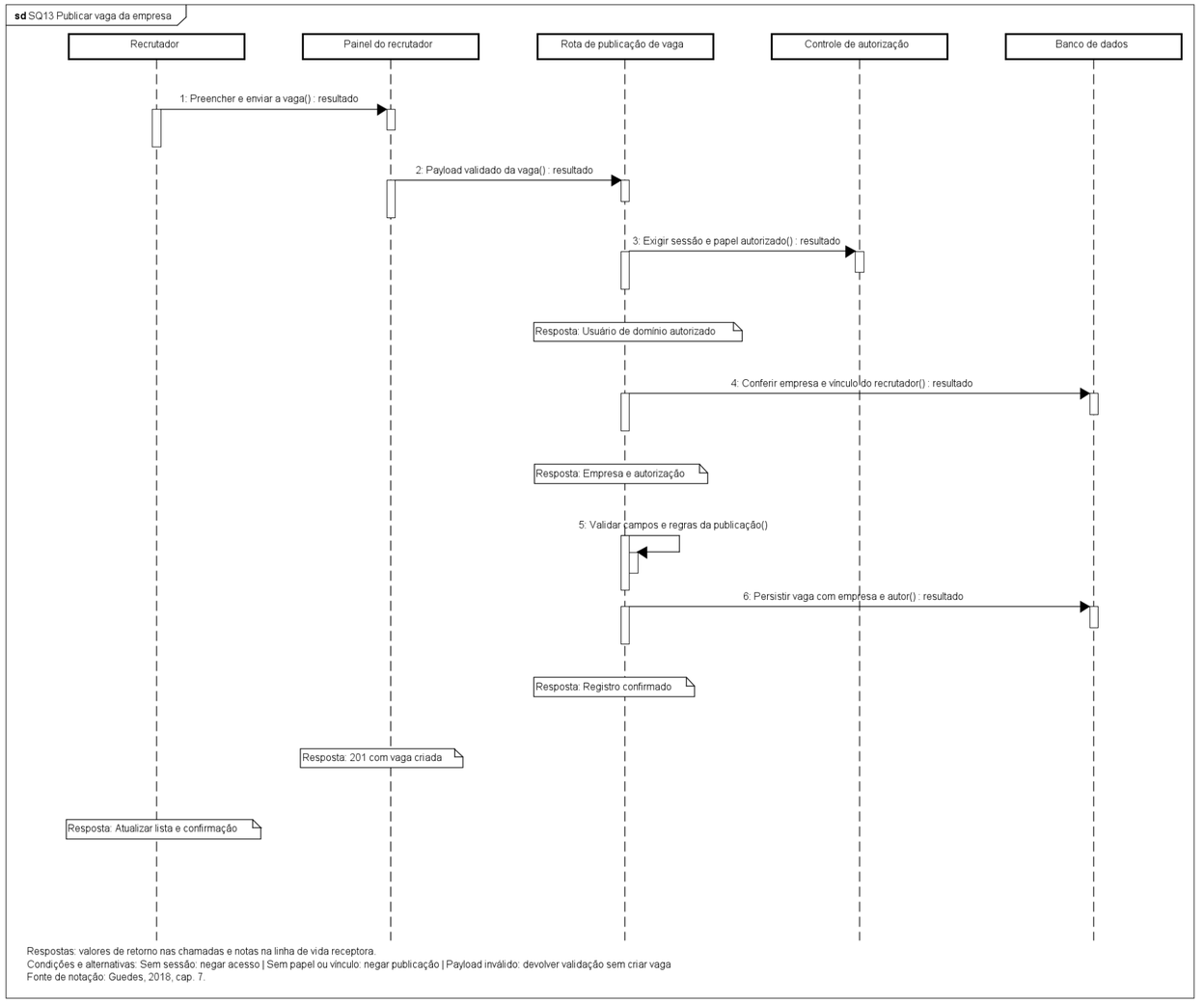
Chaves e vínculos da seleção



Visão relacional reduzida: 5 tabelas e vínculos principais. CP = chave primária. CE = chave estrangeira.
Todos os campos e relações auxiliares permanecem no dicionário técnico do banco.

Sequência completa: publicar vaga

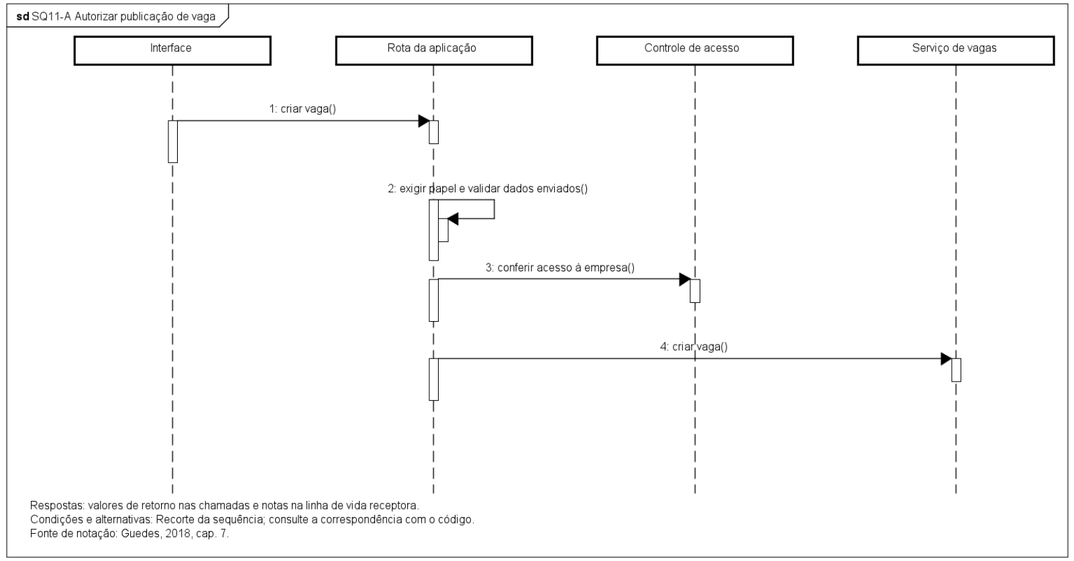
Da autorização à confirmação da interface



Autorizar publicação de vaga

SQ11-A — sequência e código correspondente

Interação



Implementação

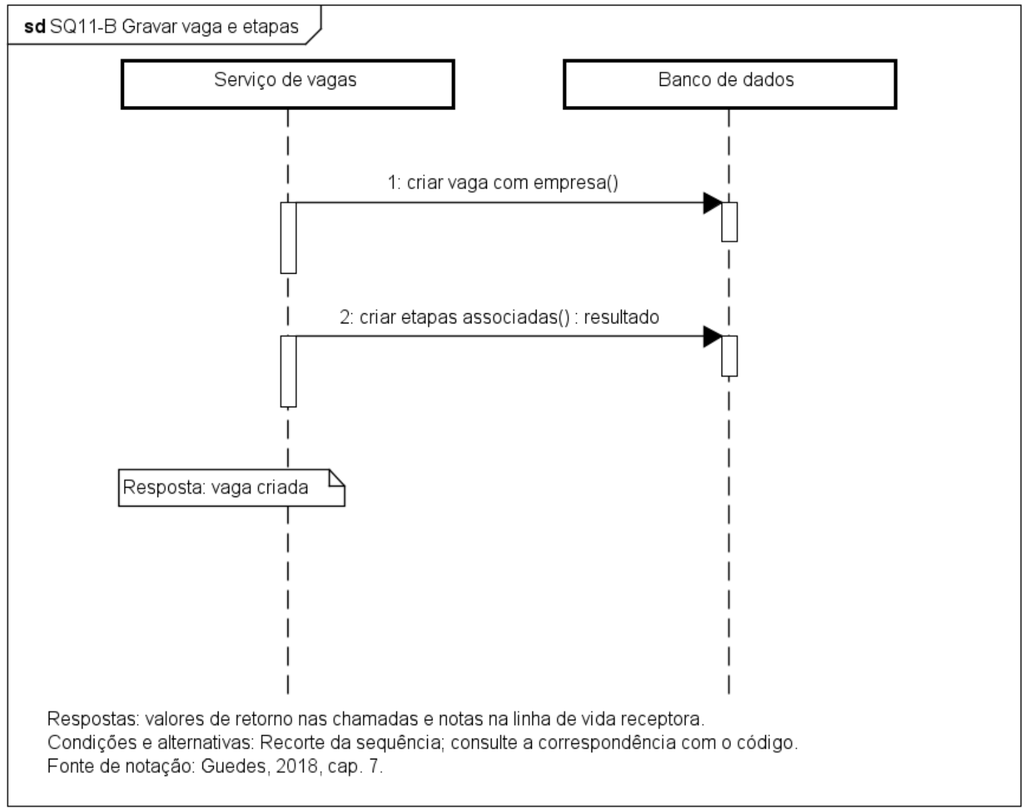
```
72  export const POST = apiHandler(async (req) => {
73    const user = await requireRole(['RECRUITER', 'ADMIN']);
74    const body = await req.json();
75    const parsed = createJobSchema.parse(body);
76    await requireCompanyAccess(parsed.company_id, user);
77
78    const job = await JobService.createJob({
79      companyId: parsed.company_id,
80      title: parsed.title,
81      description: parsed.description,
    ...
84      modality: parsed.modality,
85      location: parsed.location,
86      contractType: parsed.contract_type,
87      salaryMin: parsed.salary_min,
88      salaryMax: parsed.salary_max,
89      requirements: parsed.requirements,
90      benefits: parsed.benefits,
91      stages: parsed.stages,
92    });
93
94    return NextResponse.json({
95      success: true,
```

src/app/api/jobs/route.ts Linhas 72–95

Gravar vaga e etapas

SQ11-B — sequência e código correspondente

Interação



Respostas: valores de retorno nas chamadas e notas na linha de vida receptora.
Condições e alternativas: Recorte da sequência; consulte a correspondência com o código.
Fonte de notação: Guedes, 2018, cap. 7.

Implementação

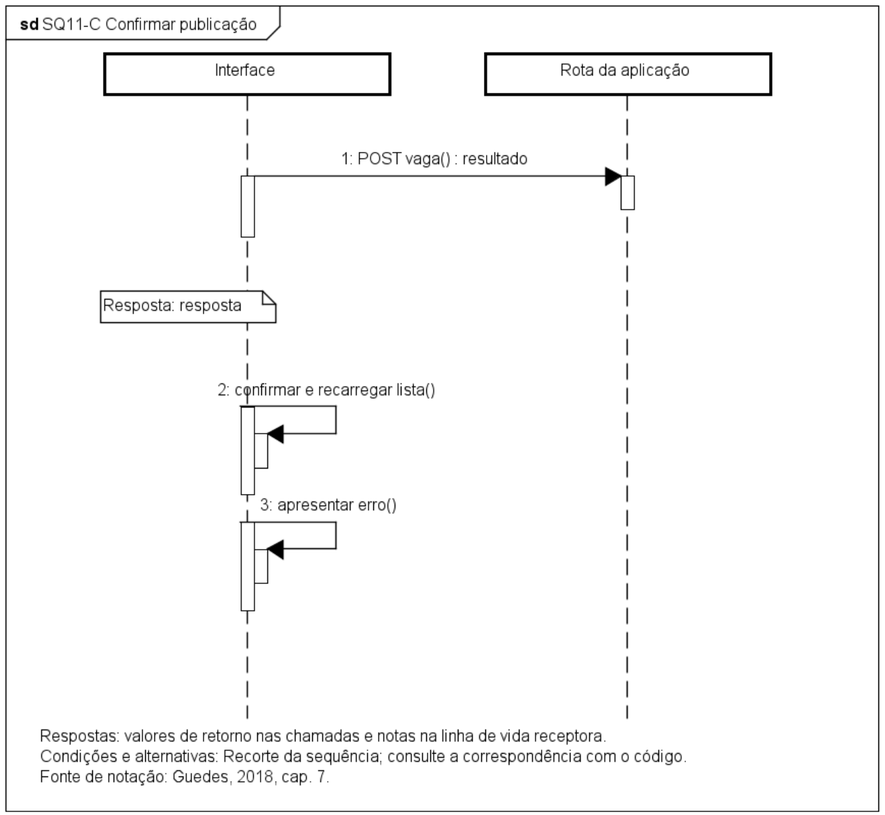
```
218      const job = await prisma.job.create({
219        data: {
220          company_id: companyId,
221          title,
222          description,
223          level,
224          technologies,
225          modality,
226          location,
227          contract_type: contractType,
     ...
236              type: st.type,
237              order: st.order,
238              challenge_id: st.challengeId,
239              description: st.description,
240            })),
241          },
242        },
243        include: {
244          stages: true,
245          company: true,
246        },
247      });
```

src/services/job.service.ts Linhas 218–247

Confirmar publicação

SQ11-C — sequência e código correspondente

Interação



Implementação

```
 378        if (res.ok) {
 379          setSuccessMsg('Vaga publicada com sucesso! As etapas do processo já estão
disponíveis.');
 380          setJobTitle('');
 381          setJobDescription('');
 382          loadJobs(); // Instantly update jobs list
 383          setTimeout(() => setSuccessMsg(null), 4000);
 384        } else {
 385          const errData = await res.json().catch(() => null);
 386          setErrorMsg(errData?.message || errData?.error || 'Erro ao publicar
vaga.');
 387        }
 388      } catch (err) {
 389        console.error(err);
 390        setErrorMsg('Erro de conexão ao publicar vaga.');
 391      } finally {
 392        setCreatingJob(false);
 393      }
 394    };
```

src/app/recruiter/RecruiterContent.tsx Linhas 378–394

Demonstração prática

Guilherme Alves de Campos Publicação de vaga

Preparação

Deixar a conta de recrutador autenticada e vinculada à empresa de demonstração. Usar /recruiter com empresa cadastrada previamente.

Execução

Identificar no protótipo a empresa e os dados obrigatórios da vaga. Publicar a vaga e mostrar a confirmação na interface.

Resultado esperado

O servidor exige papel de recrutador ou administrador e acesso à empresa. A vaga é persistida como aberta e recebe suas etapas de recrutamento.

Conferência

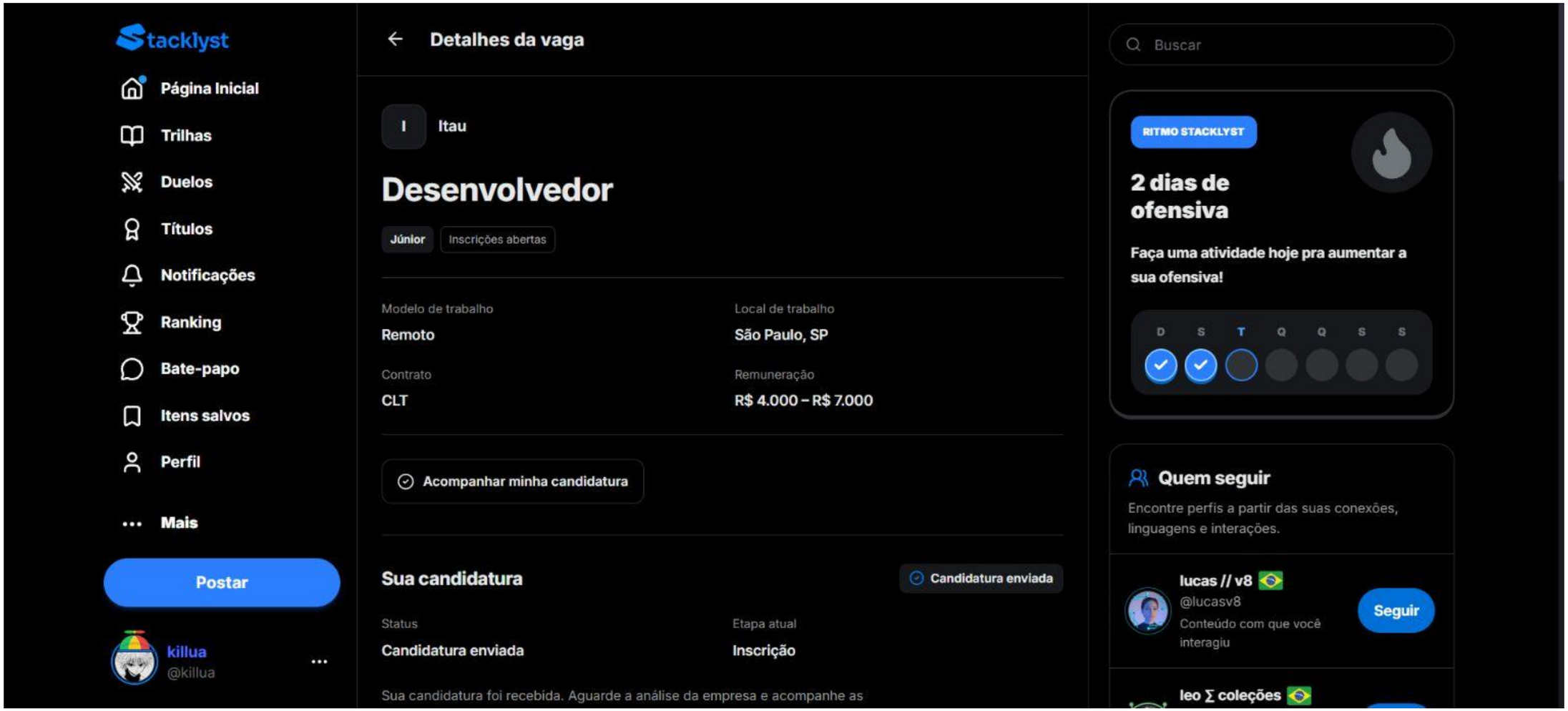
Localizar a vaga na listagem atualizada e observar as etapas associadas. Abrir /jobs e a página da vaga para preparar a continuidade da demonstração.

Continuidade

A vaga criada e sua primeira etapa já estão disponíveis para a candidatura do terceiro bloco.

Paulo: Registrar Candidatura

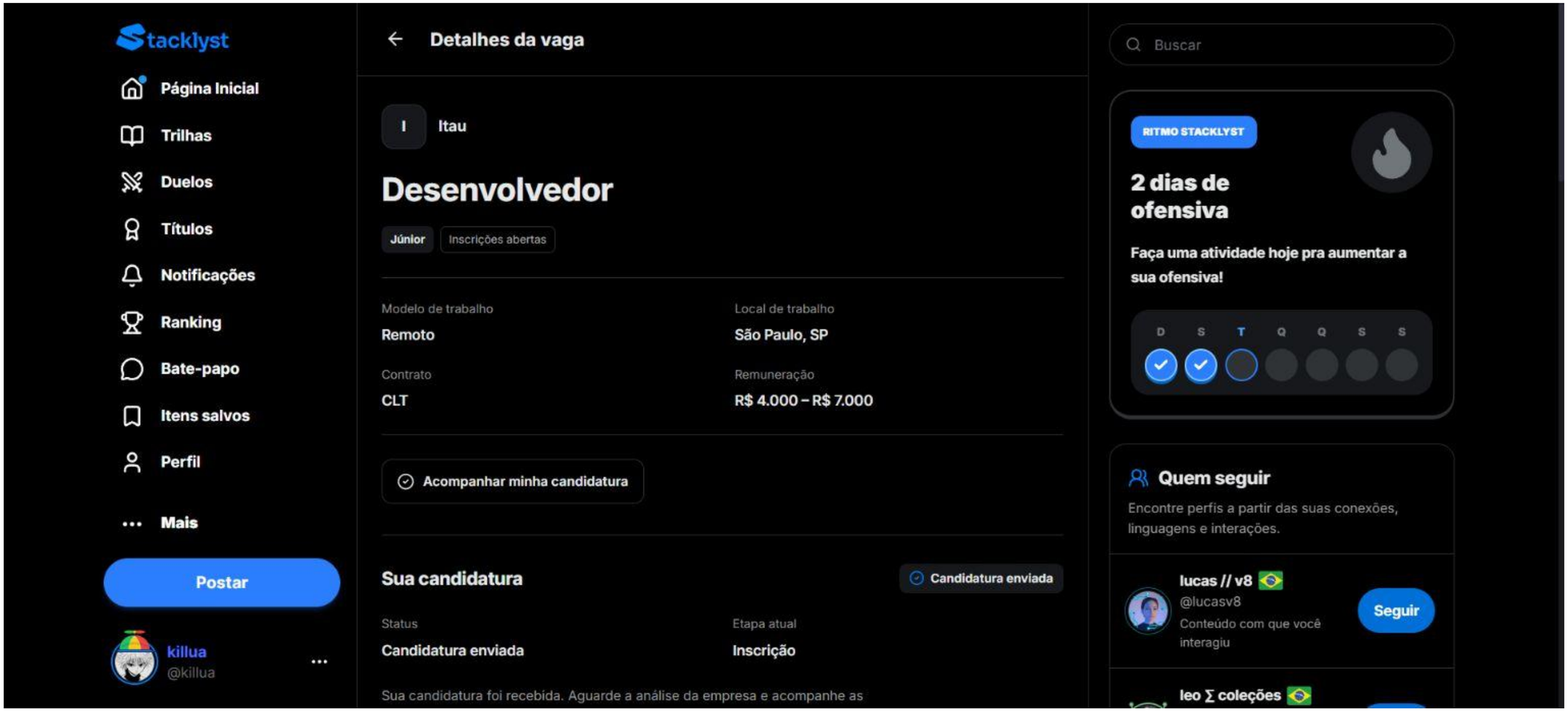
UC022: detalhe atual da oportunidade



Interface atual da plataforma, capturada em 03/11/2026

Acompanhamento da candidatura

Visão atual do candidato e situação da inscrição



Interface atual da plataforma, capturada em 03/11/2026

Requisitos e regras na interface

Candidatura e seleção

RF F40–F41

Candidatar-se e acompanhar a movimentação no funil.

RNF Q01, Q03, Q06

Autorizar o gestor, preservar dados privados e registrar auditoria.

RN R76–R80

Vaga aberta, candidatura única e vínculos compatíveis antes da alteração.

Registrar Candidatura e Gerenciar Candidaturas

Fluxo principal e alternativas da mesma vista geral

UC022 Registrar Candidatura

1. O usuário consulta uma vaga e solicita a candidatura.
2. O servidor verifica a sessão, a existência da vaga e sua situação aberta.
3. O sistema verifica se já existe candidatura do mesmo usuário à vaga.
4. O sistema cria a candidatura com sua situação inicial e os vínculos corretos.

Alternativa

A1 — Candidatura já existente: o sistema informa a inscrição anterior e não cria outra.

UC025 Gerenciar Candidaturas

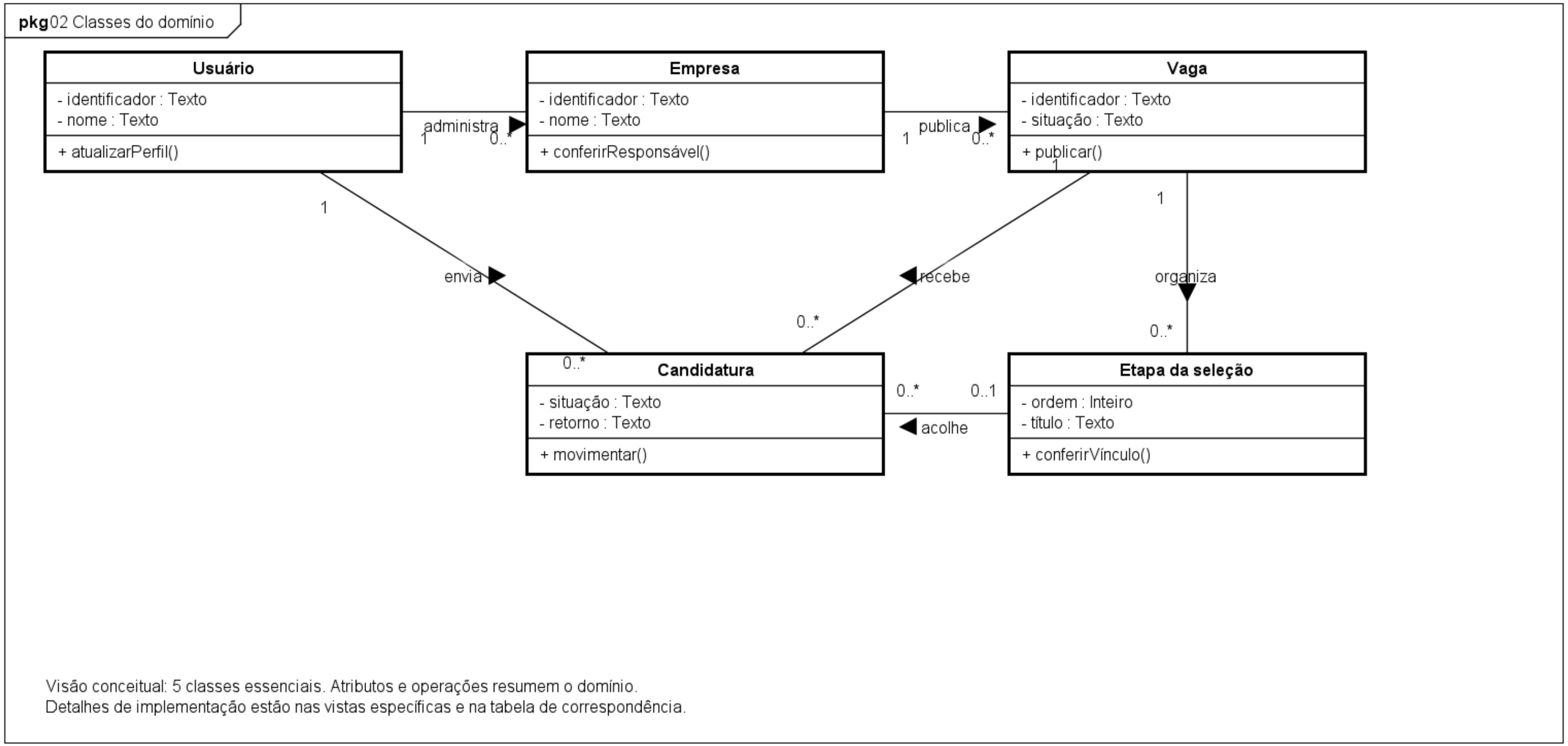
1. O gestor abre o painel de candidaturas de uma vaga autorizada.
2. O sistema valida o vínculo com a empresa e apresenta os candidatos.
3. O gestor seleciona uma candidatura e informa etapa, situação ou feedback.
4. O servidor verifica a compatibilidade entre candidatura, vaga e etapa.

Alternativa

A1 — Nova etapa, situação ou feedback: o sistema valida os vínculos, grava a mudança e registra a comunicação elegível.

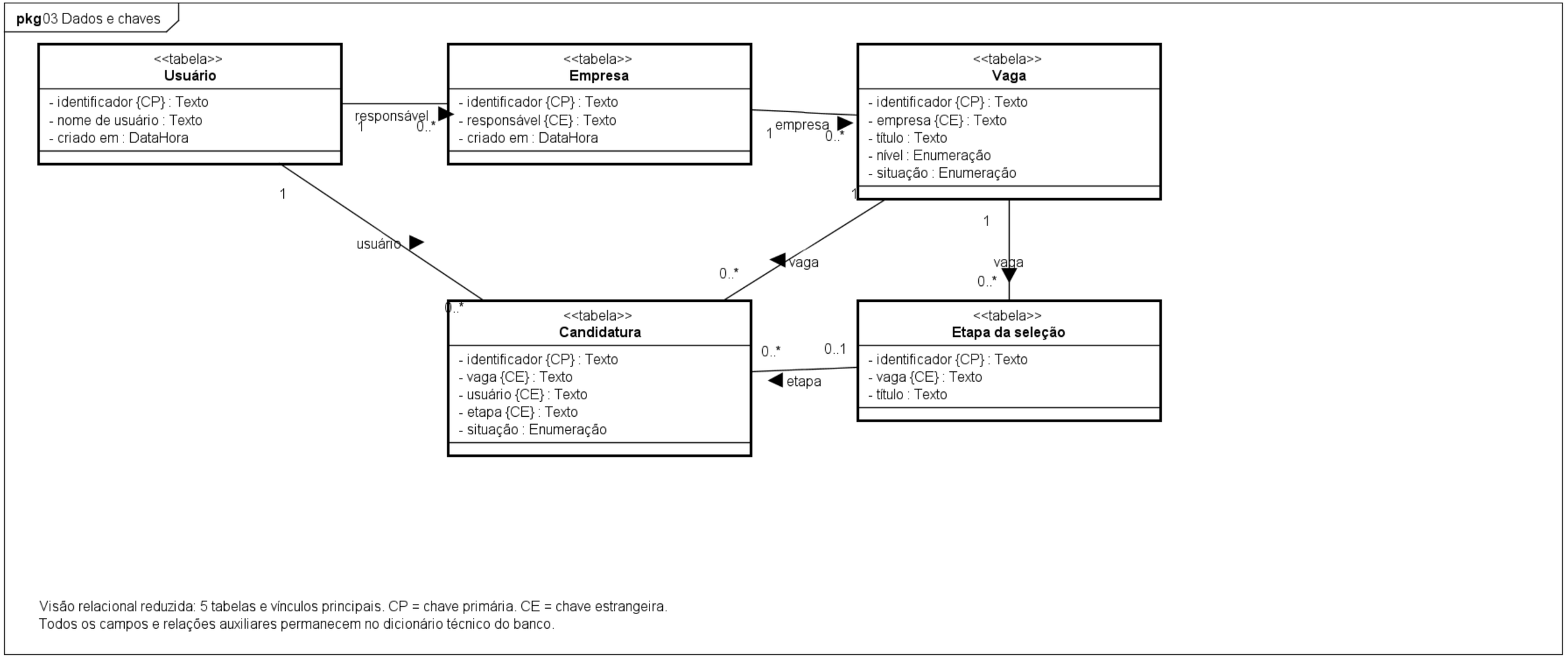
Classes da candidatura e do retorno

Vínculos e responsabilidades



Banco de dados da candidatura

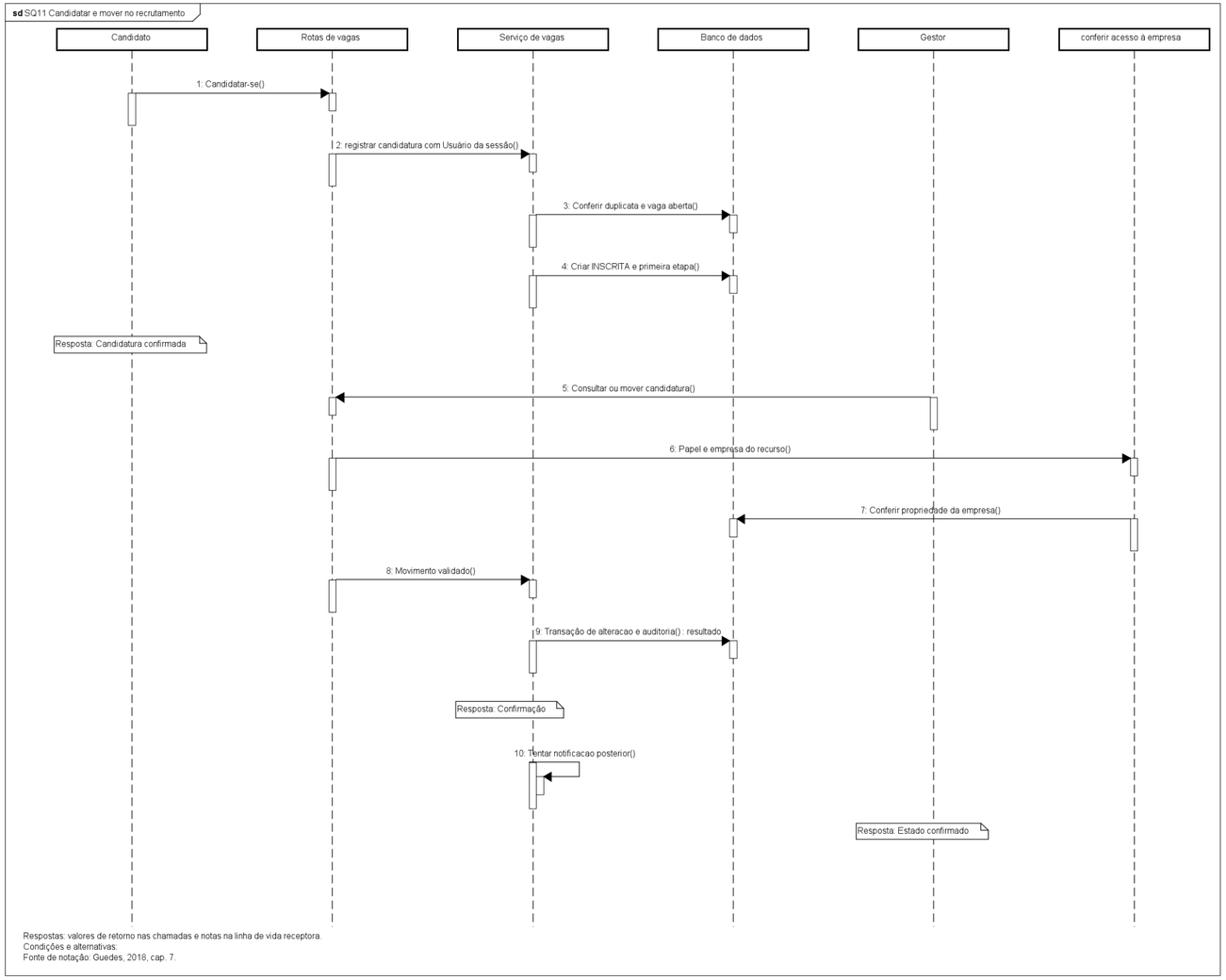
Integridade e persistência



Visão relacional reduzida: 5 tabelas e vínculos principais. CP = chave primária. CE = chave estrangeira.
Todos os campos e relações auxiliares permanecem no dicionário técnico do banco.

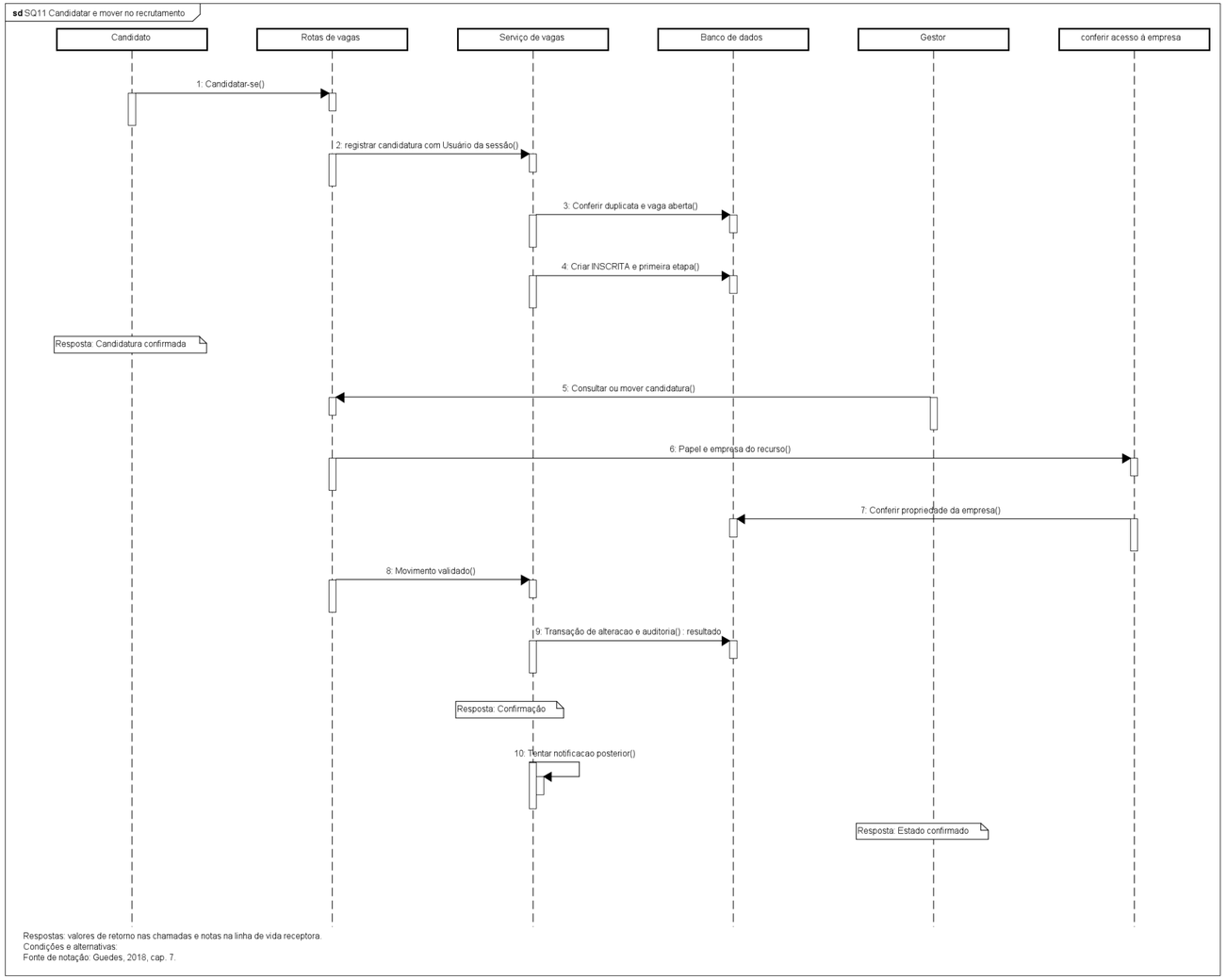
Sequência completa: candidatura

Inscrição e encaminhamento da seleção



Sequência completa: movimentar candidatura

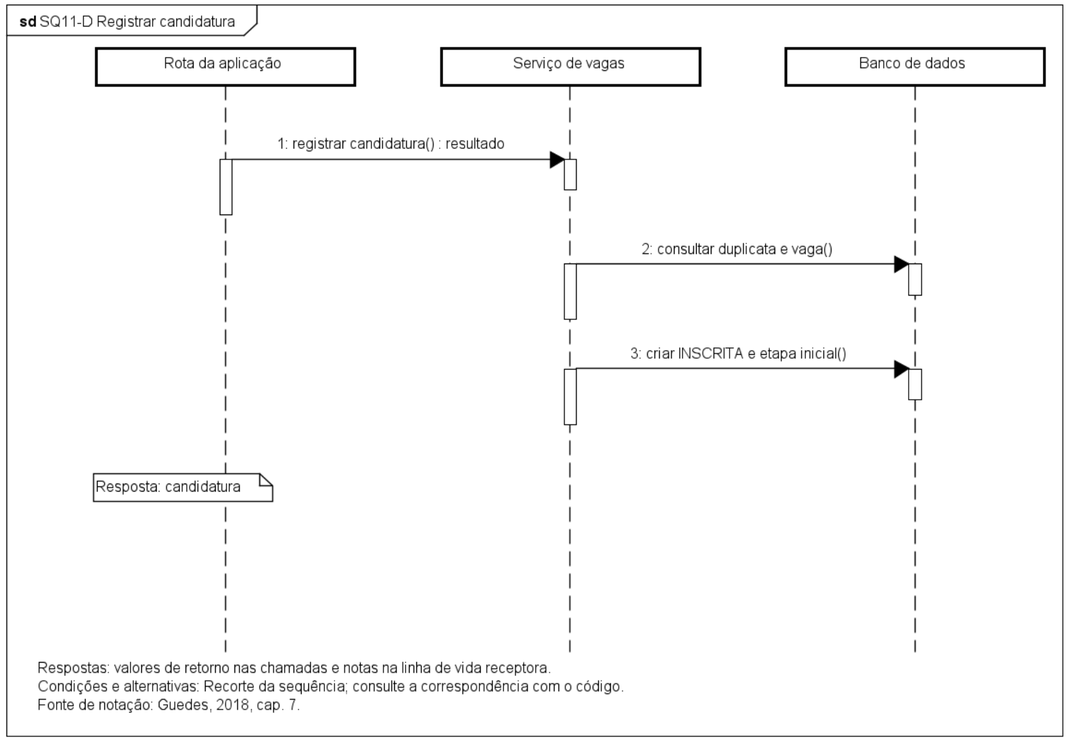
Atualização autorizada e notificação



Registrar candidatura

SQ11-D — sequência e código correspondente

Interação



Implementação

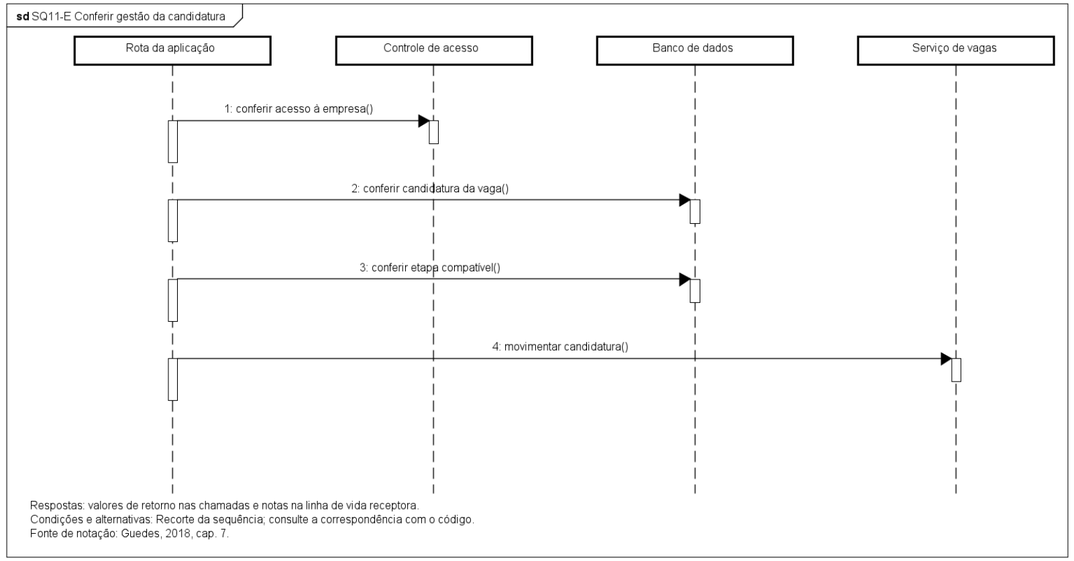
```
282      const firstStage = job.stages[0];
283
284      const application = await prisma.jobApplication.create({
285        data: {
286          job_id: jobId,
287          user_id: userId,
288          stage_id: firstStage?.id,
289          status: 'APPLIED',
290        },
291        include: {
292          stage: true,
293          job: { include: { company: true } },
294        },
295      });
296
297      logger.info('User applied for job', { userId, jobId, applicationId:
application.id });
298      return application;
299    },
300
301    /**
302     * Advance or update candidate application status in recruitment pipeline.
303     */
304    async updateApplicationStage(params: {
```

src/services/job.service.ts Linhas 282–304

Conferir gestão da candidatura

SQ11-E — sequência e código correspondente

Interação



Implementação

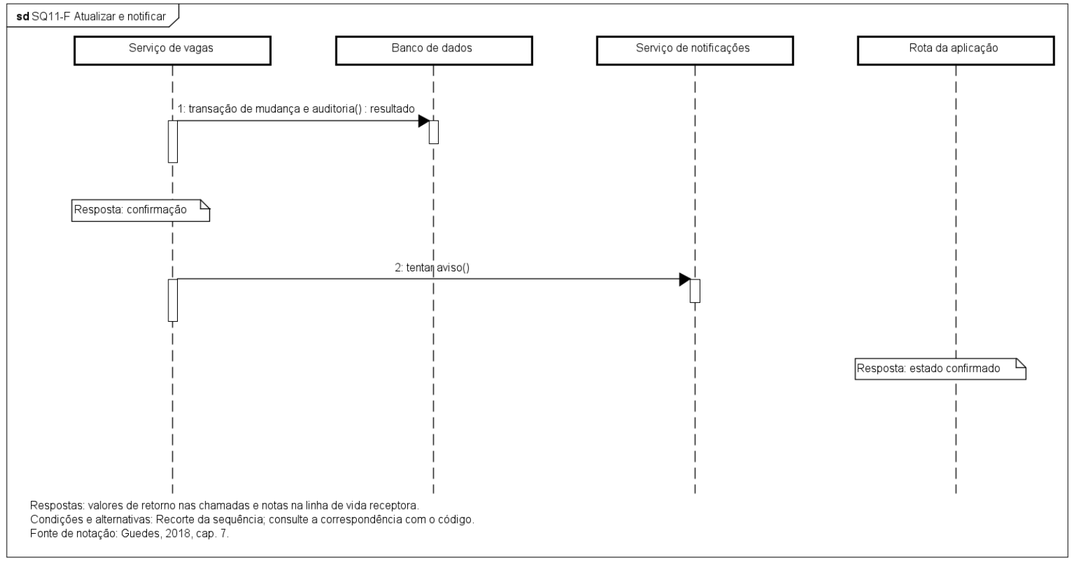
```
30  export const PATCH = apiHandler(async (req, { params }) => {
31    const user = await requireRole(['RECRUITER', 'ADMIN']);
32    const { id } = await params;
33    const job = await prisma.job.findUniqueOrThrow({ where: { id } });
34    await requireCompanyAccess(job.company_id, user);
35    const body = await req.json();
36    const parsed = updateStageSchema.parse(body);
37    const application = await prisma.jobApplication.findFirst({
38      where: { id: parsed.application_id, job_id: id },
39    });
    ...
51      status: parsed.status,
52      feedback: parsed.feedback,
53      technicalScore: parsed.technical_score,
54      actorId: user.id,
55    });
56
57    return NextResponse.json({
58      success: true,
59      message: 'Etapa do candidato atualizada com sucesso!',
60      application: updatedApplication,
61    });
62  });
```

src/app/api/jobs/[id]/route.ts Linhas 30–62

Atualizar e notificar

SQ11-F — sequência e código correspondente

Interação



Implementação

```
314      const application = await prisma.$transaction(async (tx) => {
315        const updated = await tx.jobApplication.update({
316          where: { id: applicationId },
317          data: {
318            stage_id: stageId,
319            status,
320            feedback,
321            technical_score: technicalScore,
322          },
323          include: {
     ...
347        await NotificationService.create({
348          userId: application.user_id,
349          type: 'JOB_APPLICATION_UPDATE',
350          resourceId: application.job_id,
351          resourceType: 'JOB',
352        });
353      } catch (err) {
354        logger.error('Failed to notify job application update', { error: String(err)
});
355      }
356
357      return application;
358    },
```

src/services/job.service.ts Linhas 314–359

Demonstração prática

Paulo Henrique Leal dos Santos Candidatura, acompanhamento e retorno ao candidato

Preparação

Usar a vaga criada no bloco anterior e uma conta de candidato que ainda não se candidatou a ela. Manter duas sessões separadas, candidato e recrutador, para evitar pausas de autenticação.

Execução

Na sessão do candidato, abrir a vaga e acionar Candidatar-se com Stacklyst. Mostrar a confirmação da candidatura na etapa inicial.

Resultado esperado

A candidatura referencia a vaga e o candidato e começa na primeira etapa disponível. Uma candidatura repetida é rejeitada e vagas fechadas não aceitam inscrição.

Conferência

Na sessão do recrutador, carregar as candidaturas da vaga e localizar o mesmo candidato. No seletor da candidatura, alterar a situação de Inscrito para Em Análise e mostrar a confirmação. Voltar à sessão do candidato, atualizar a consulta e localizar a notificação da atualização.

Continuidade

Encerrar mostrando o percurso completo: estudar, disponibilizar uma oportunidade, candidatar-se e receber o retorno do processo.

Percurso demonstrado

Aprendizagem e seleção com continuidade

Aprendizagem

A resposta é avaliada, o progresso é registrado e a primeira conclusão válida pode conceder XP.

Oportunidade

Um responsável autorizado pública a vaga e disponibiliza as etapas da seleção.

Candidatura e retorno

O candidato ingressa no processo e acompanha a atualização feita pelo recrutador.

Referências

Modelagem, requisitos e implementação

Referência de UML

GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. Novatec, 2018.

Requisitos e qualidade

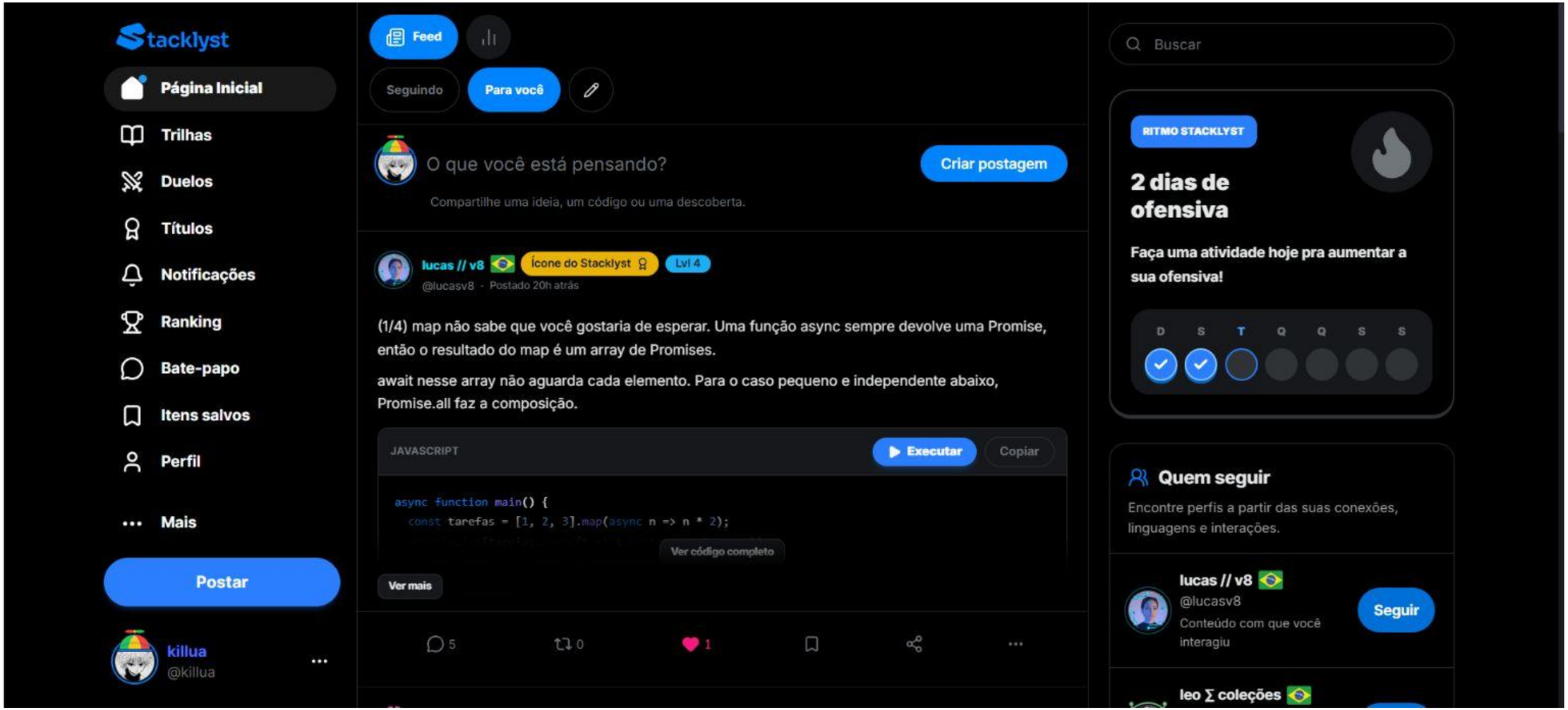
Matriz de rastreabilidade e imagens de classificação de RNF e fatores de McCall fornecidas pelo grupo.

Fontes do projeto

Documentos 01–05, catálogo com 26 casos da vista Caso de Uso Geral, projeto Astah e código-fonte da Stacklyst. Orientações do seminário fornecidas pela professora.

Página inicial e feed

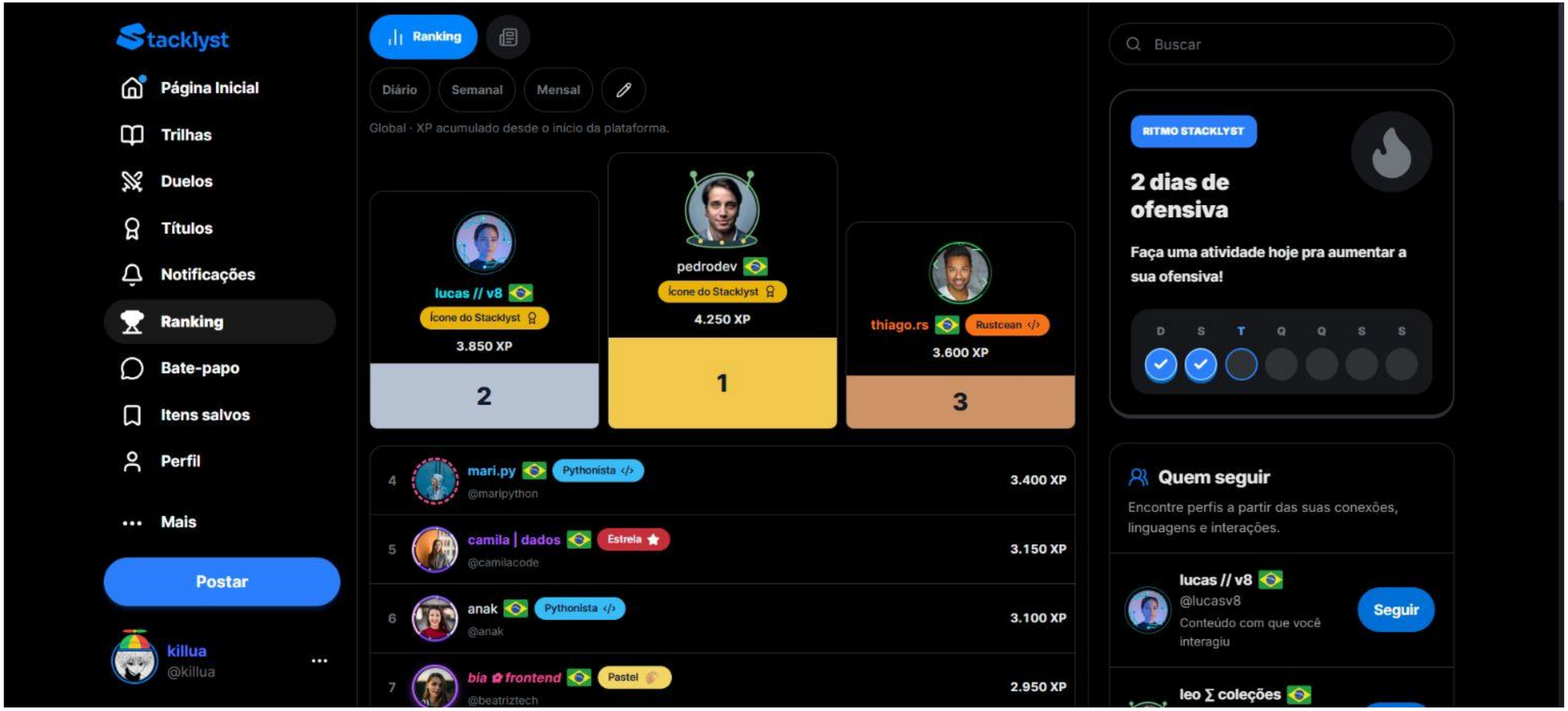
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/feed

Ranking de XP

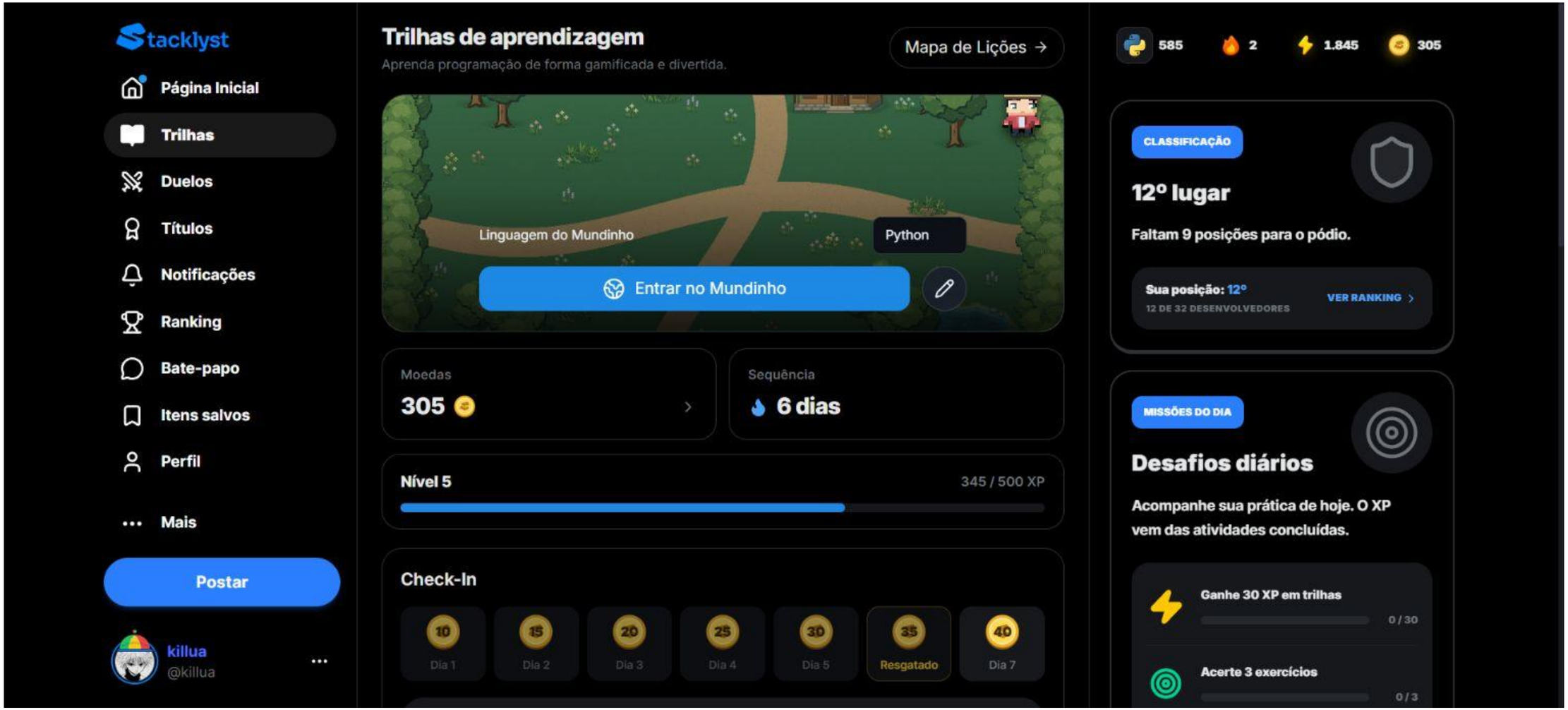
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/ranking

Trilhas de aprendizagem

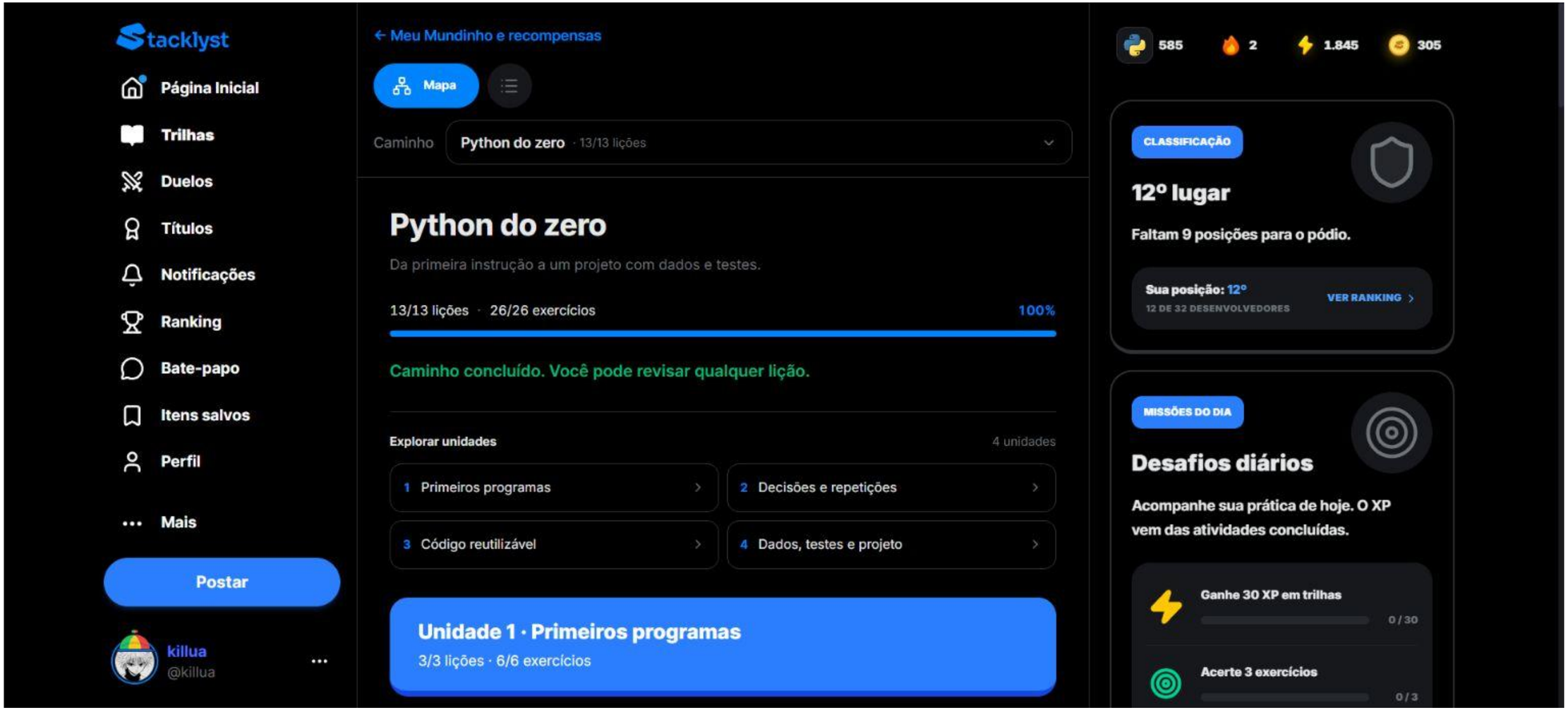
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/trails

Mapa de lições Python

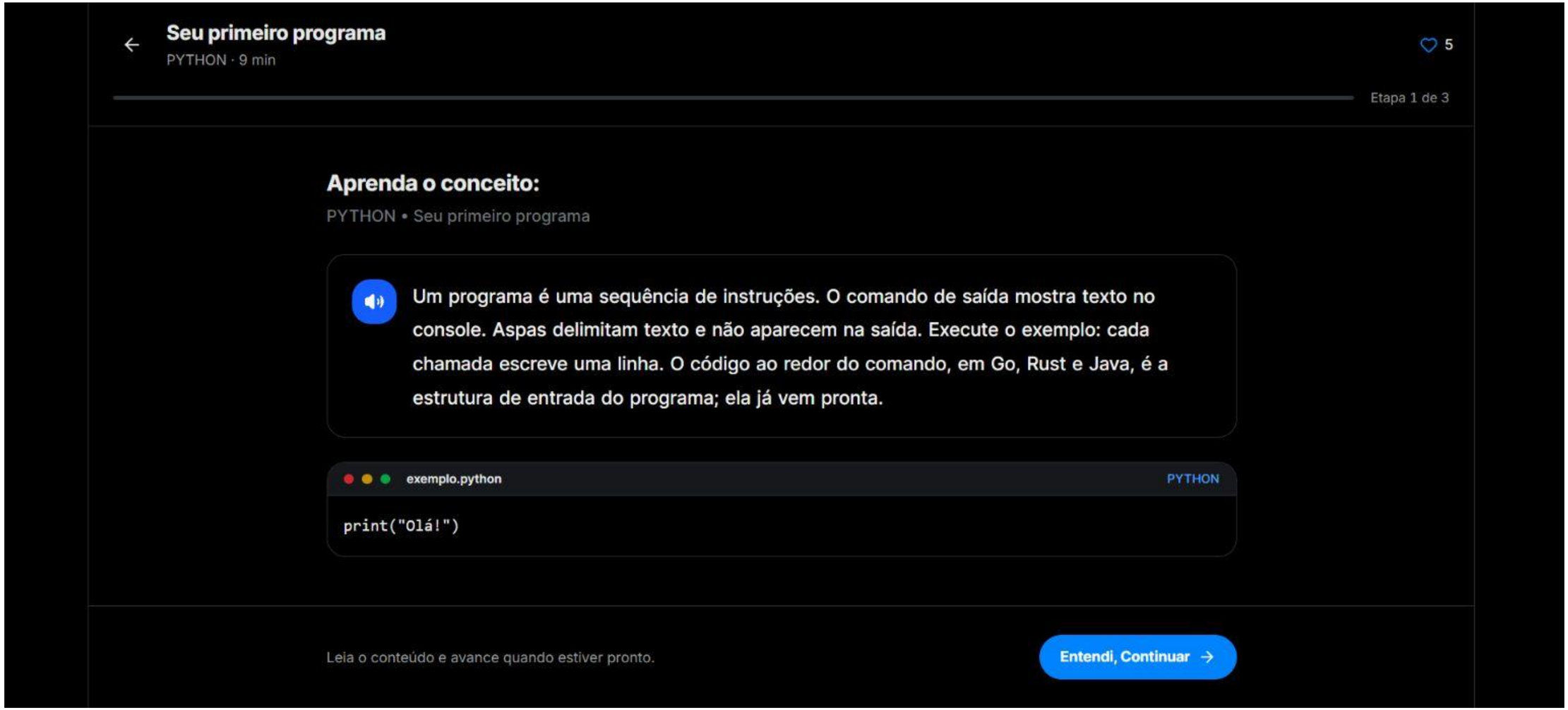
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/trails?view=map

Realizar Lição

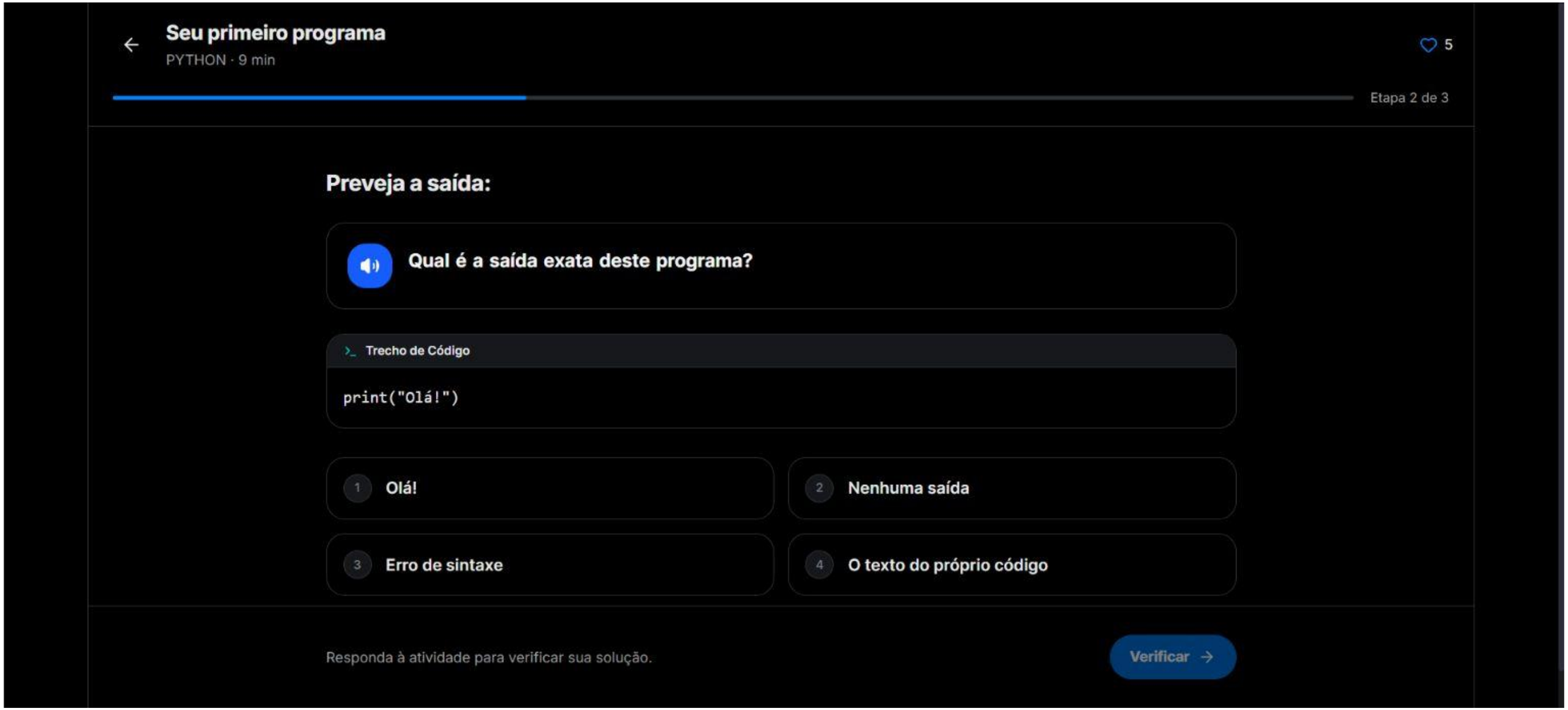
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/lesson/learn-python-first-output?returnTo=%2Ftrails%3Fview%3Dmap%26path%3Dfoundations-python%26language%3DPYTHON

Resolver exercício

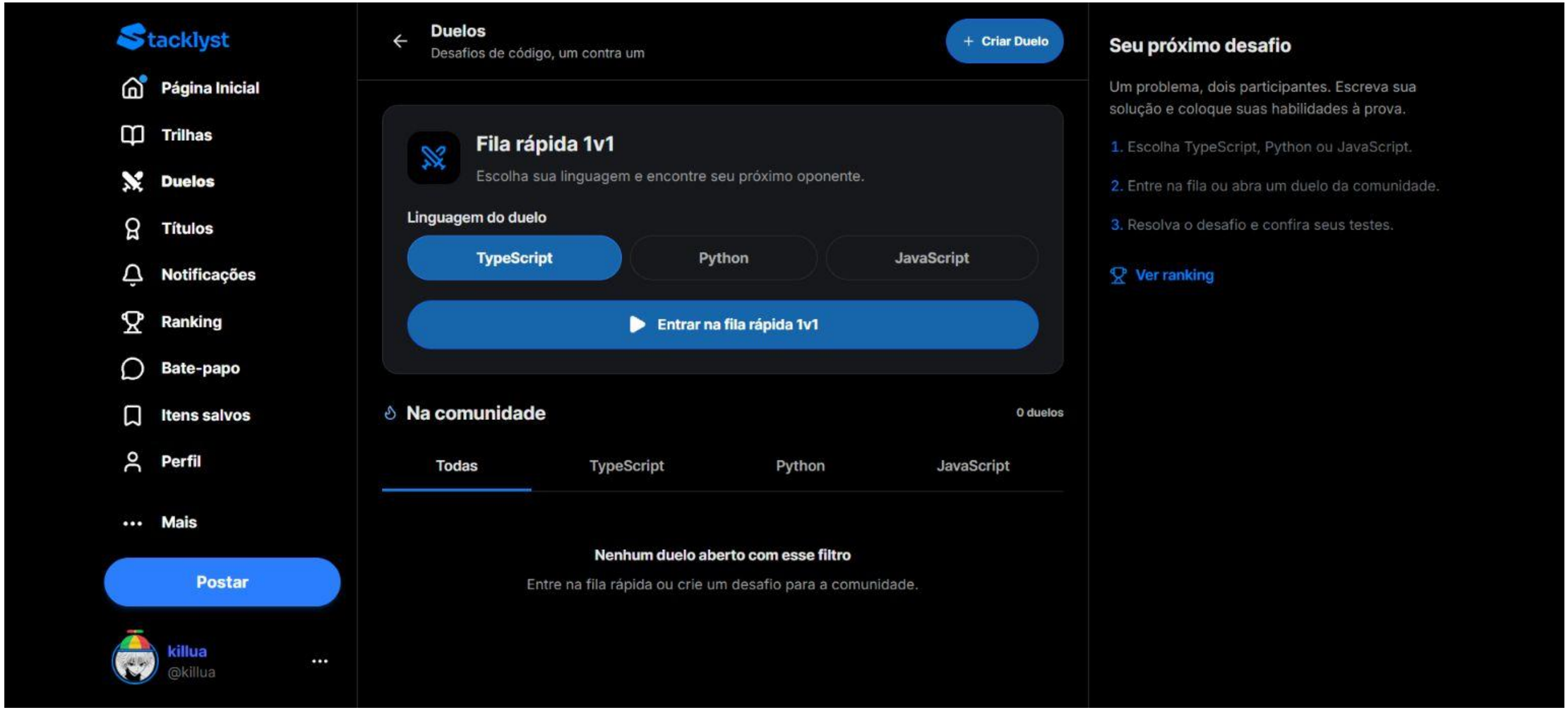
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/lesson/learn-python-first-output?returnTo=%2Ftrails%3Fview%3Dmap%26path%3Dfoundations-python%26language%3DPYTHON

Realizar Duelo

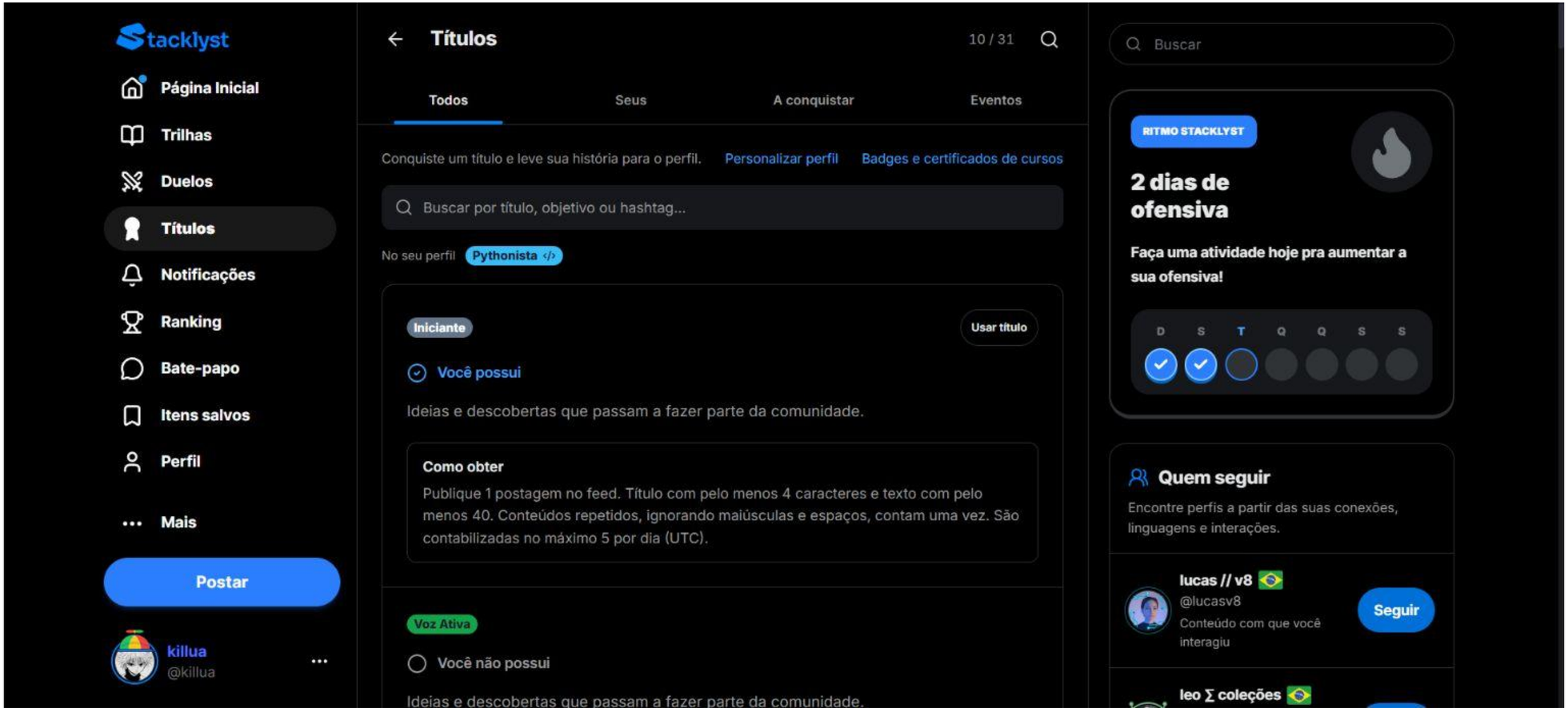
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/duels

Títulos e conquistas

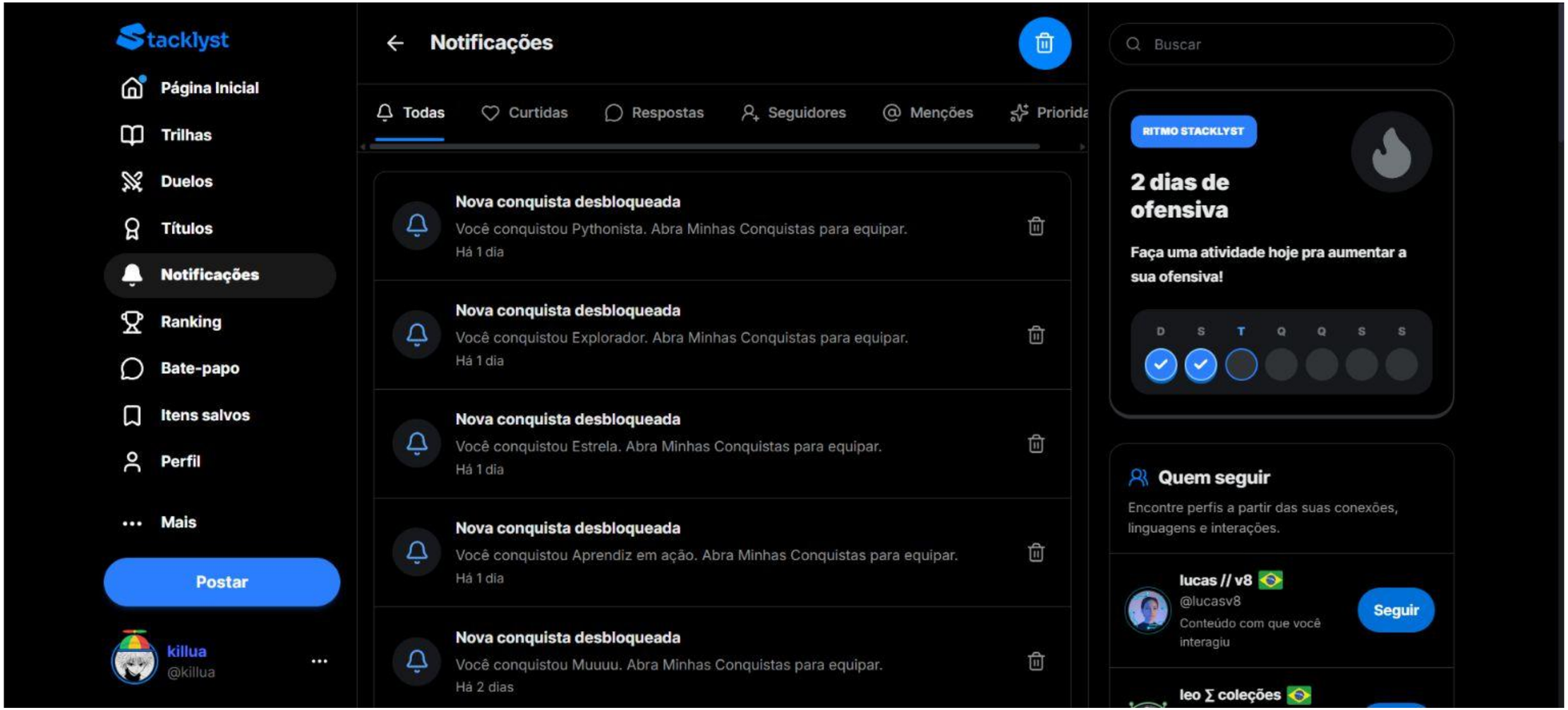
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/achievements

Notificações

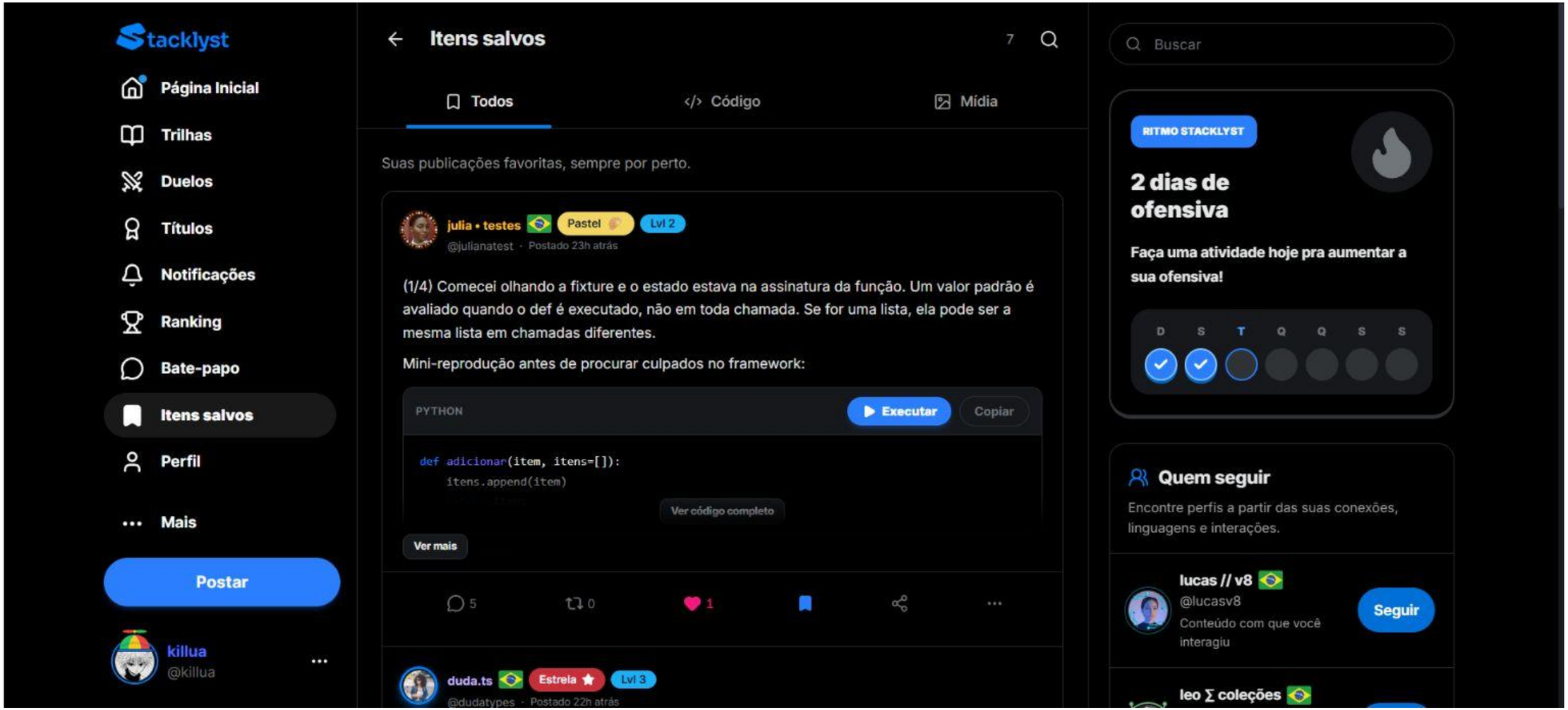
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/notifications

Itens salvos

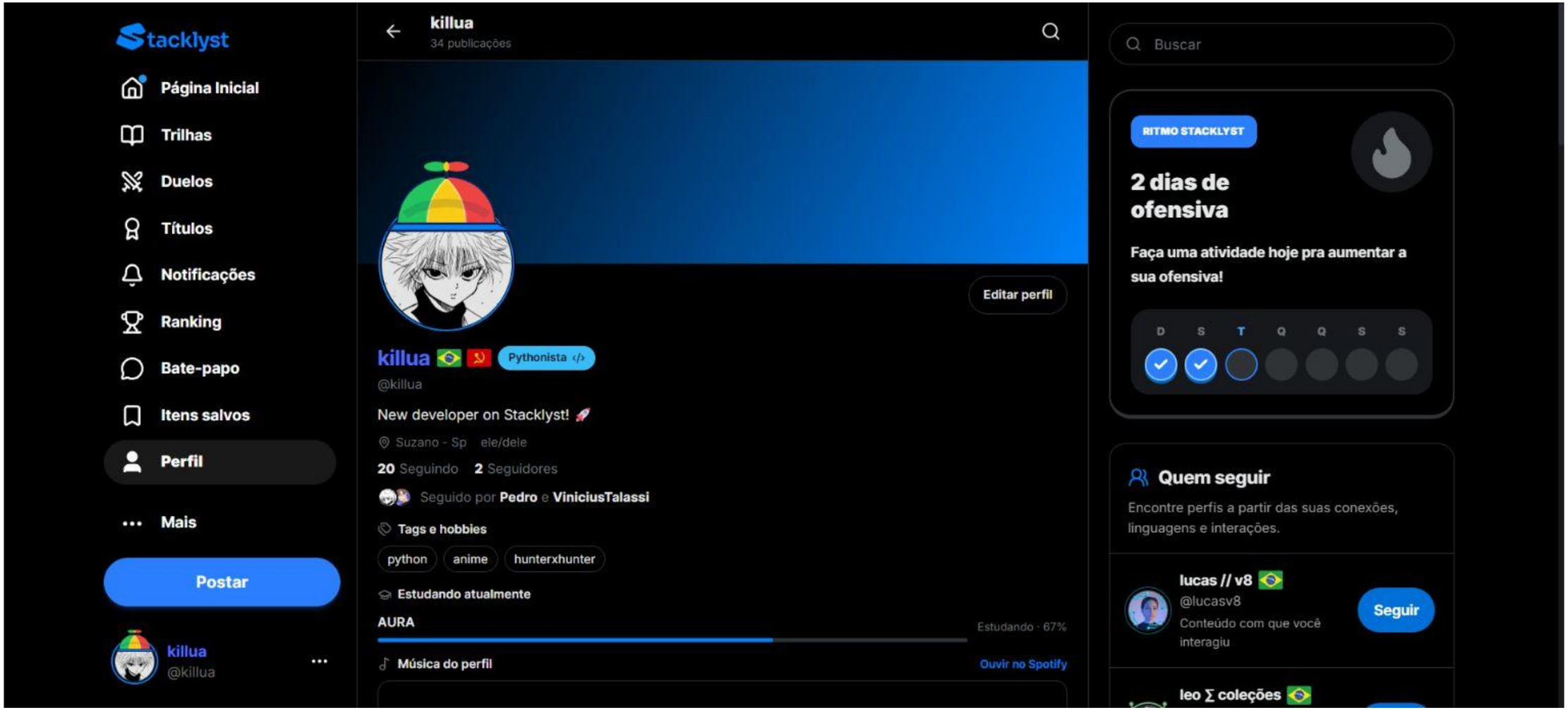
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/bookmarks

Gerenciar Perfil

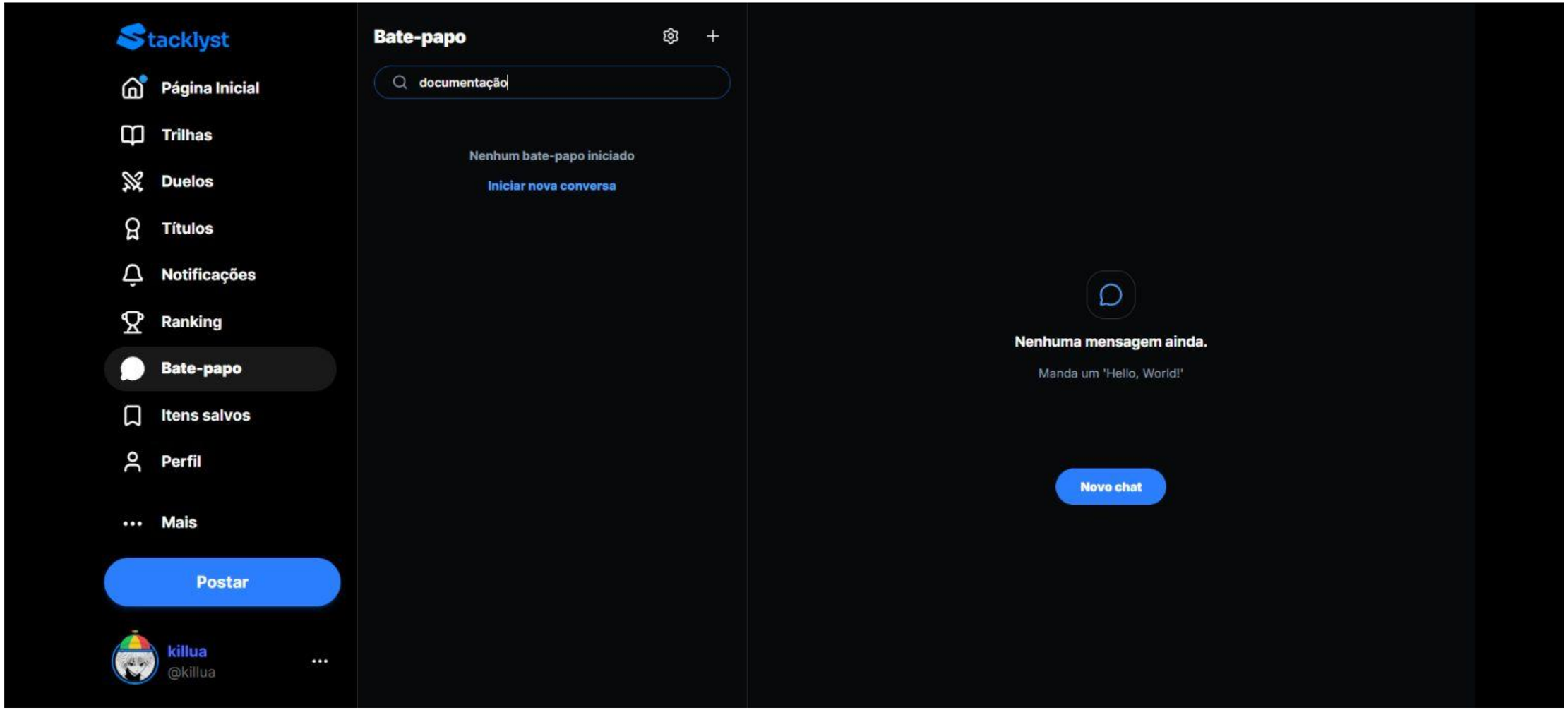
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/profile/killua

Bate-papo com busca sem resultados

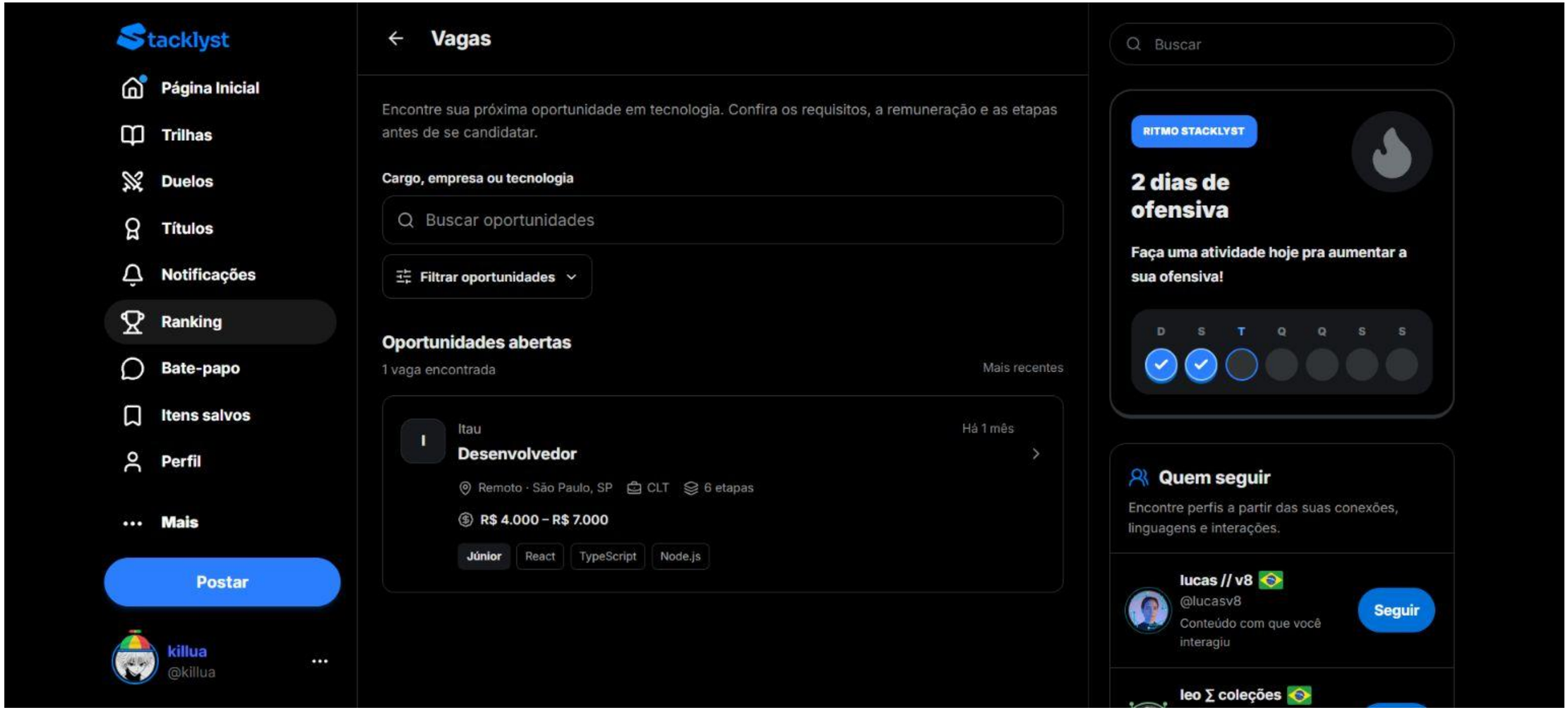
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/messages

Vagas e recrutamento

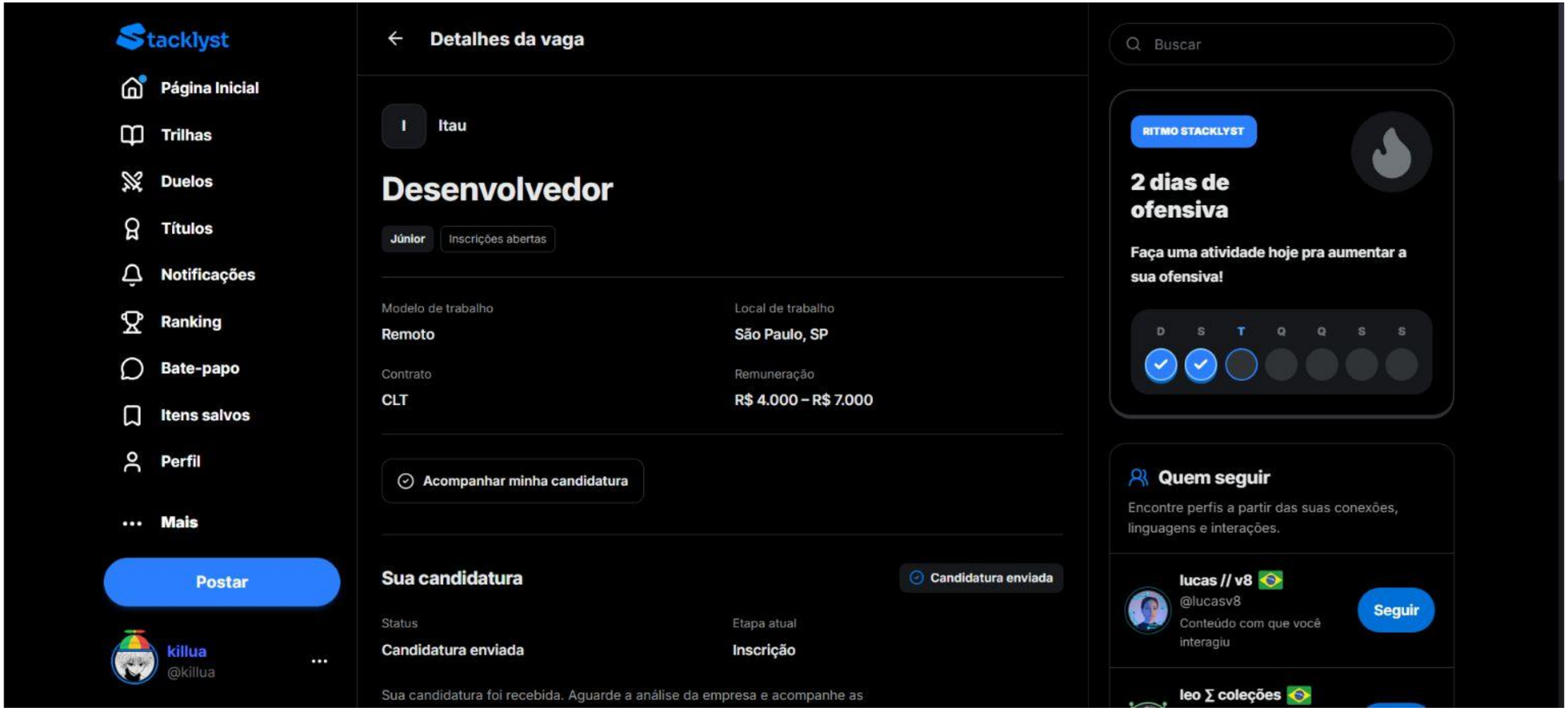
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/jobs

Detalhes da vaga e acompanhamento da candidatura

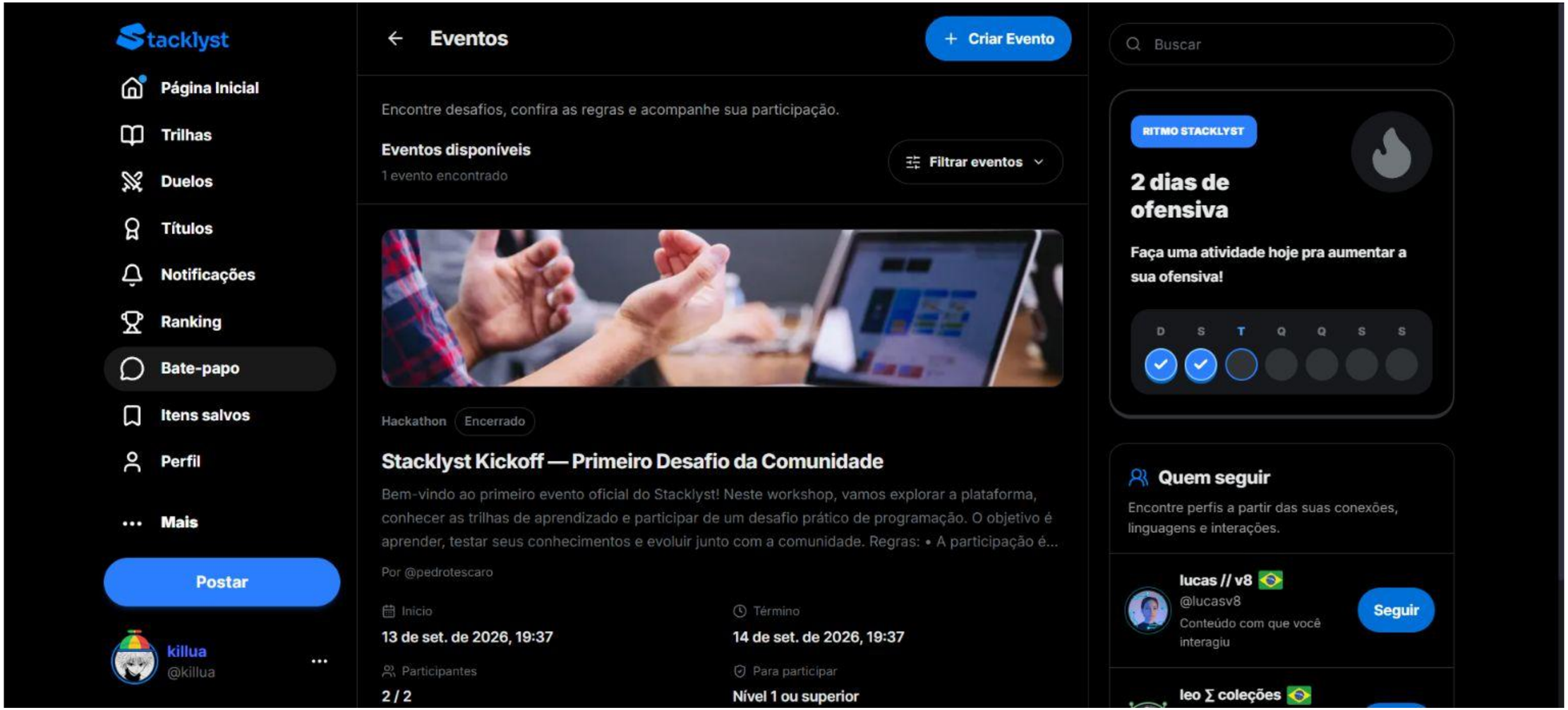
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/jobs/43827ed5-dbbc-4256-9c63-a16dbd6c9d58

Eventos e hackathons

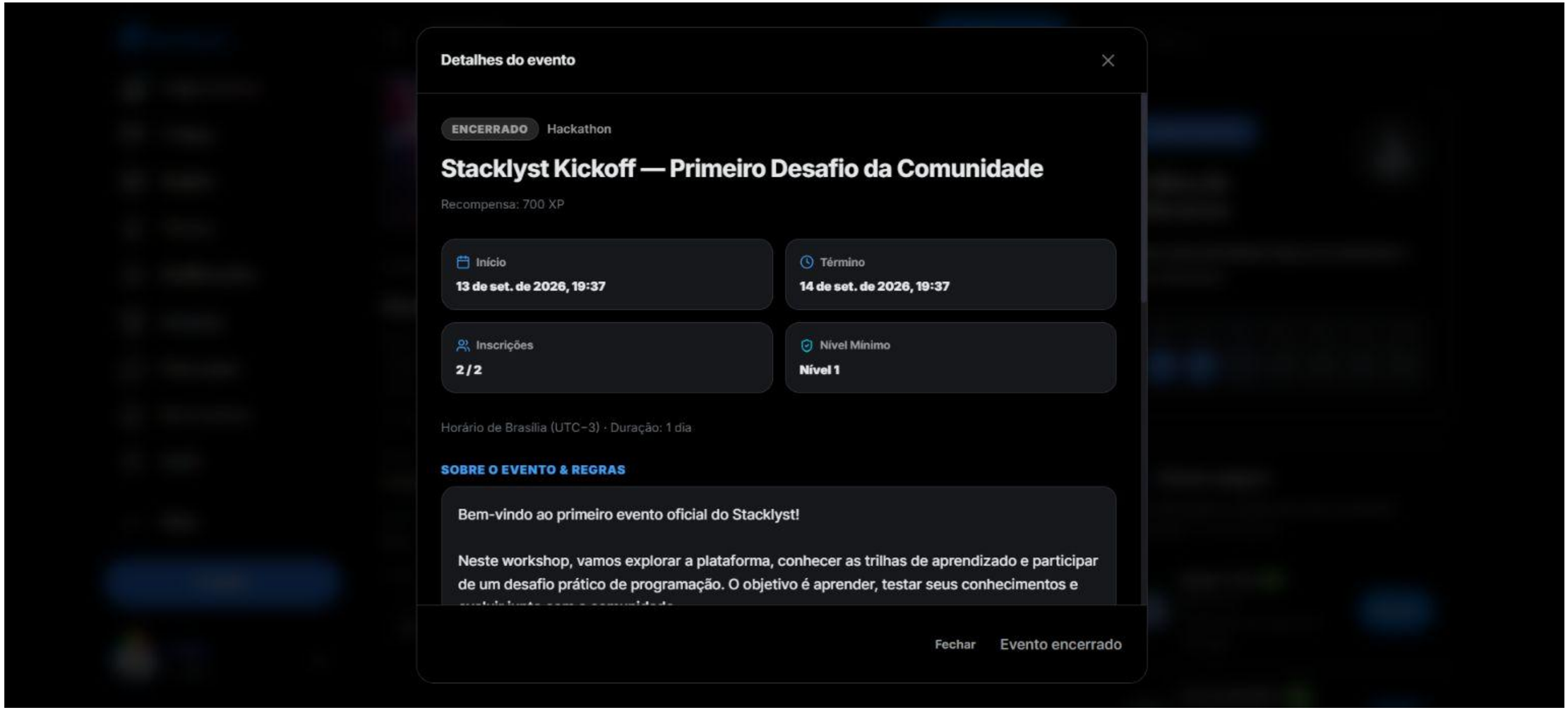
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/events

Detalhes do evento

Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/events

Avaliação de código e pedido de credenciamento

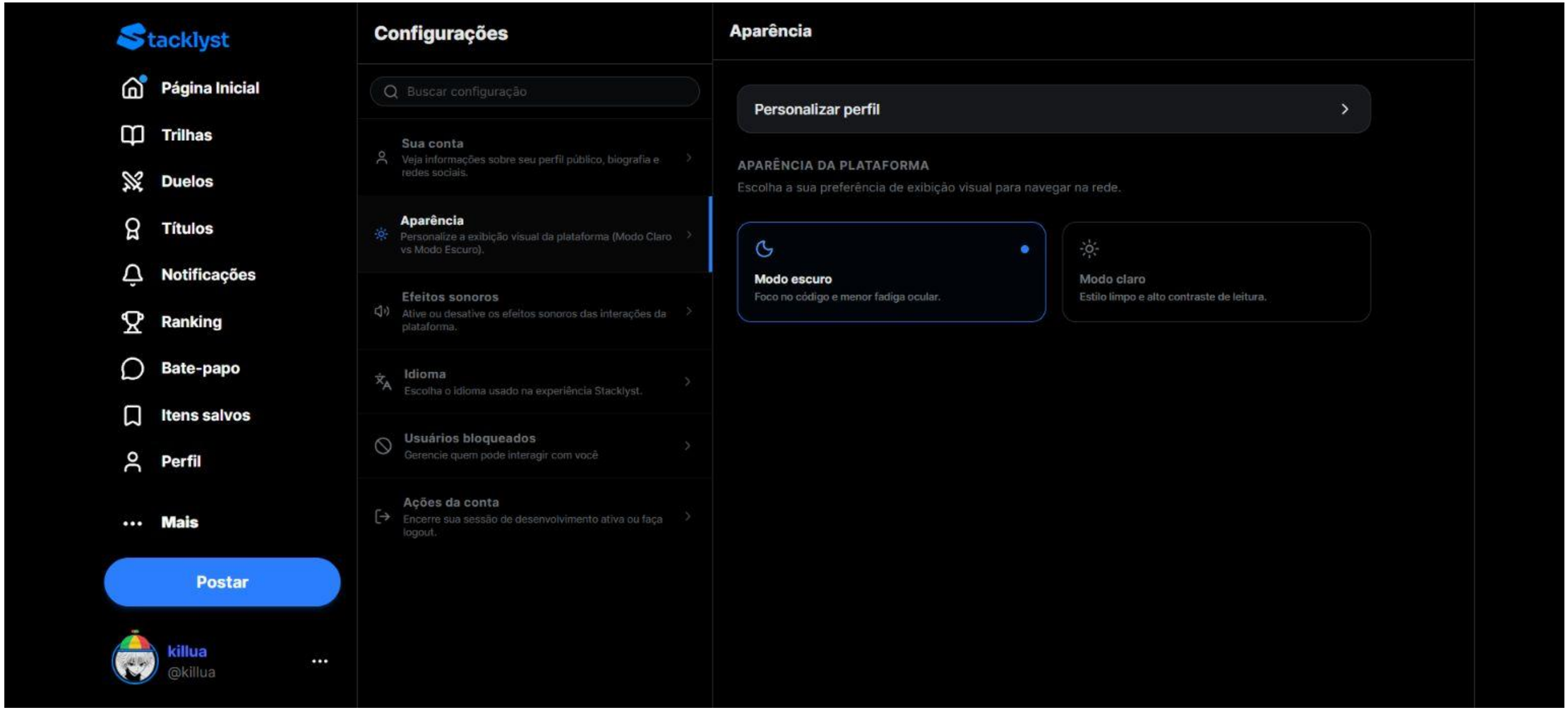
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/evaluators/apply

Configurações de aparência

Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/settings

Meu Mundinho

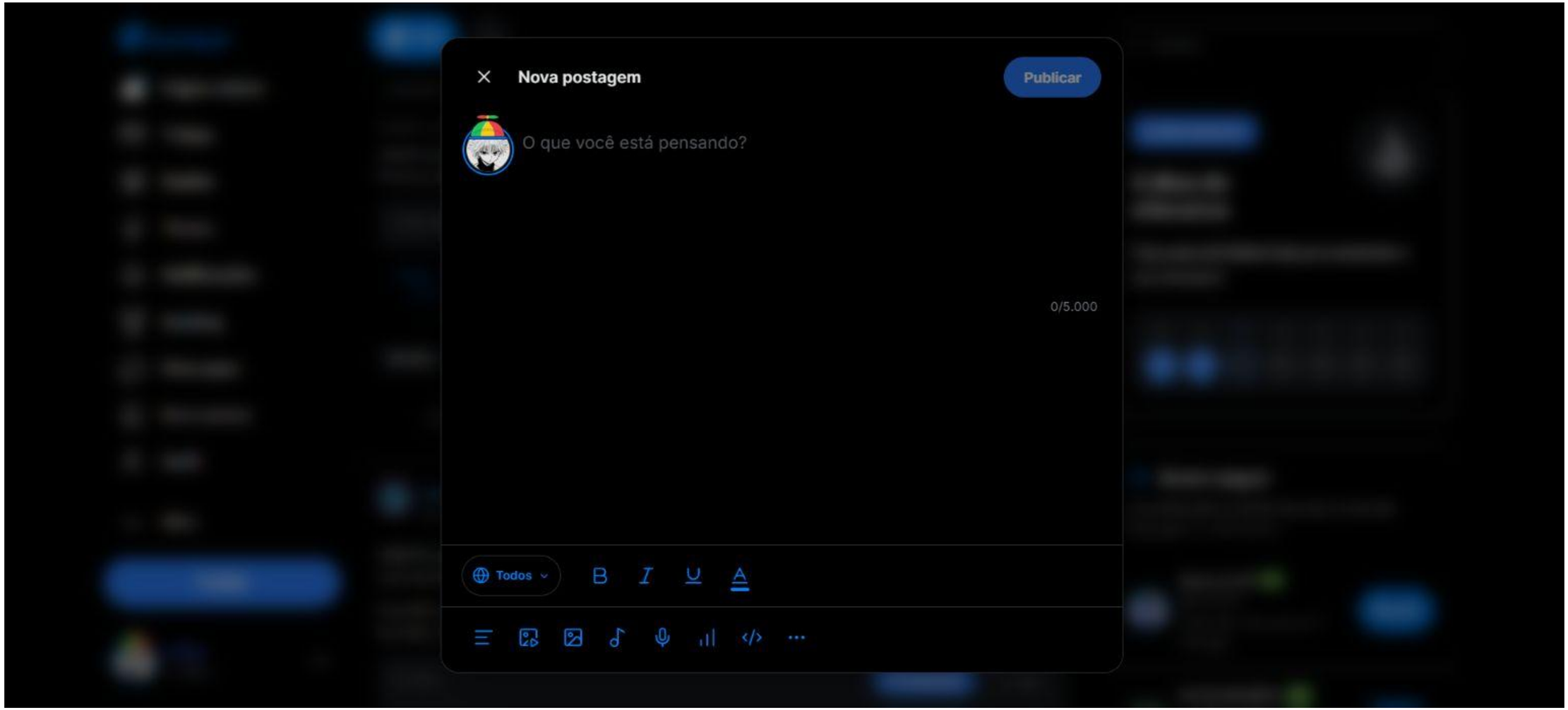
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/trails

Gerenciar Publicações: formulário

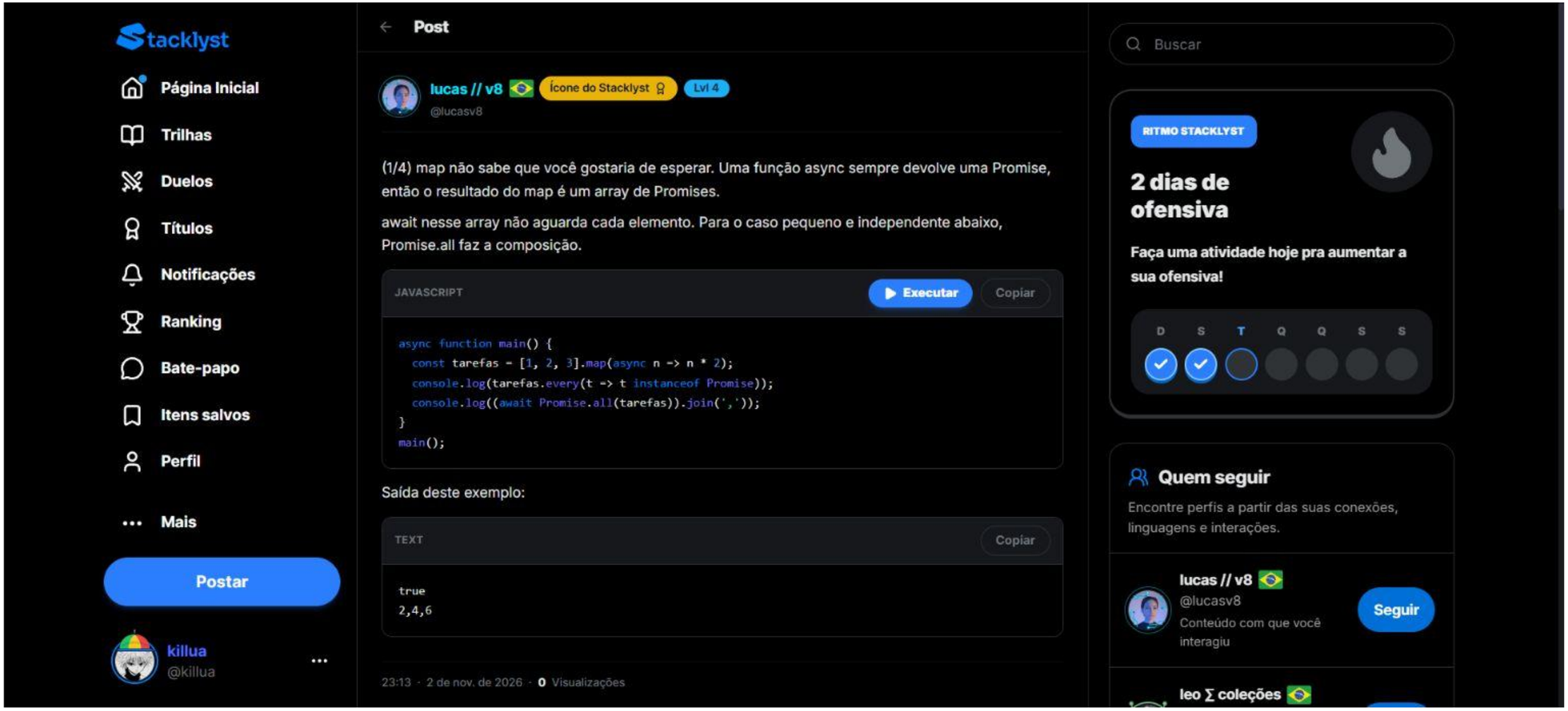
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/feed

Publicação e thread de programação

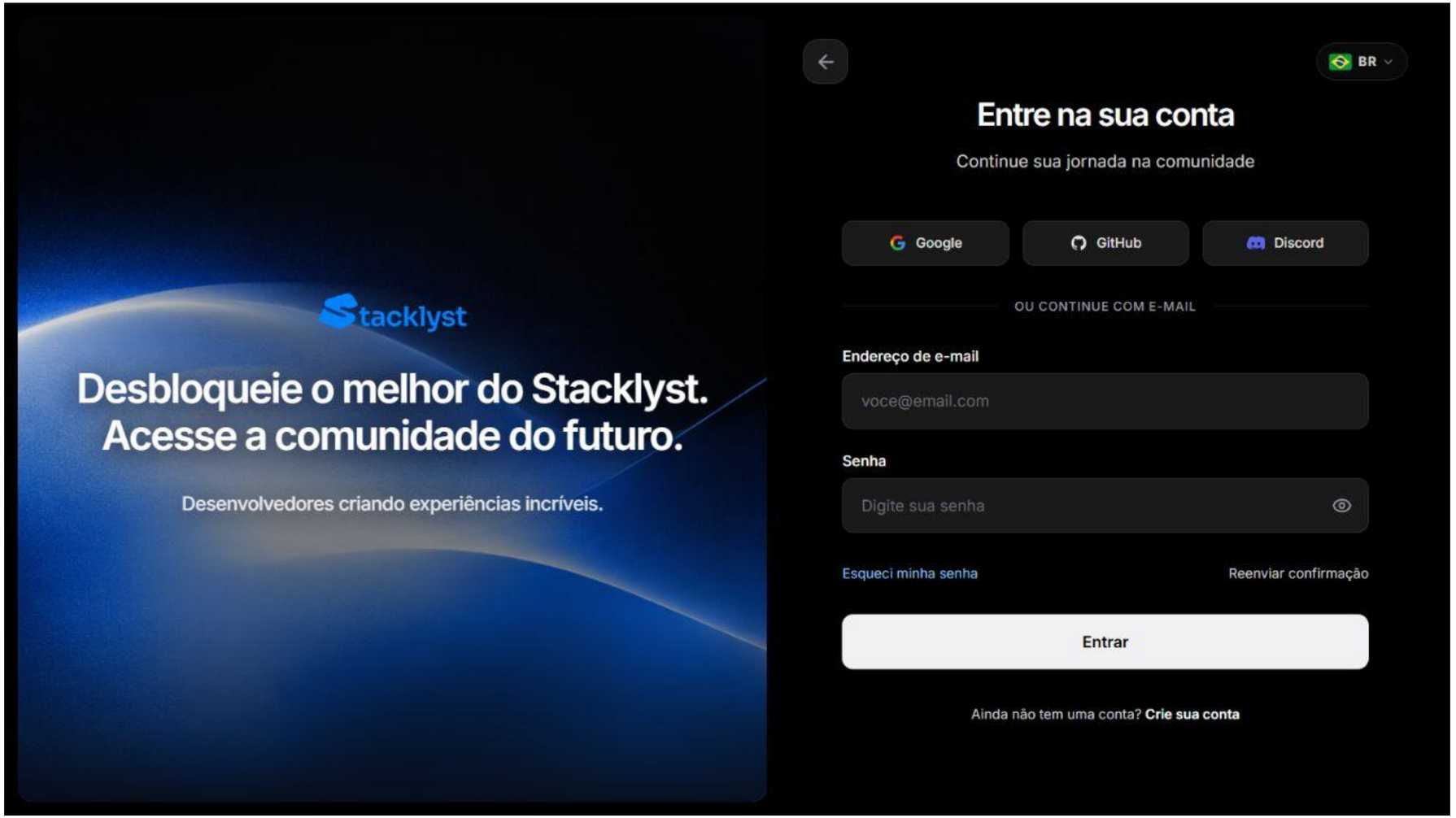
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/post/20f9b21c-f59b-4184-a0e6-0bec60c003cb

Fazer Login

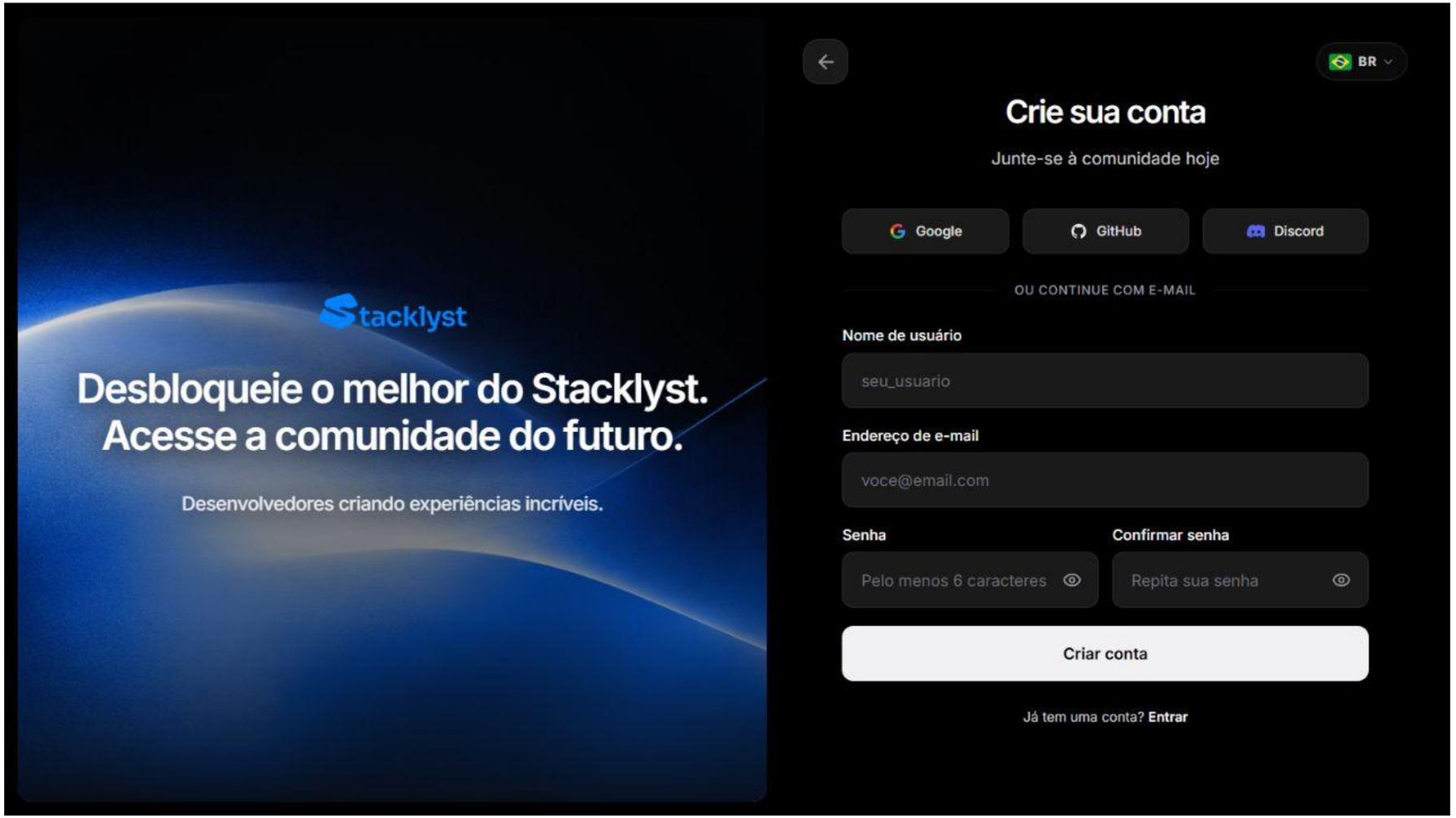
Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/login

Cadastrar Conta

Apêndice: interface atual da plataforma



03/11/2026 https://stacklyst.vercel.app/register